Advanced Text Processing

Multimedia Retrieval · Chapter 3

Multimedia Retrieval - 2026 - Uni Basel

Roger Weber

Contents

3 - Advanced Text Processing 1
Tokenization and Normalization 2
The naive tokenizer and where it breaks 2
Modern tokenization with nltk and spaCy 2
Case, Unicode, and accents 3
Sentence segmentation 4
Word boundaries are not always spaces 4
Language detection 5
Where this sits in modern retrieval 6
Stemming and Lemmatization 7
Stop words: what to keep out 7
Stemming: rule-based reduction 7
Lemmatization: dictionary-based reduction 9
Multi-lingual comparison 10
Where this sits in modern retrieval 11
Phrases and Compounds 12
Bi-grams, tri-grams, phrases 12
Naive frequency and the stop-word problem 12
Pointwise mutual information 13
Likelihood ratio 14
PMI or LHR: which to use 15
Choosing a threshold and indexing strategy 15
Compounds 17
Splitting compounds 17
Where this sits in modern retrieval 18
Query Understanding 20
Word relationships: synonyms and homonyms 20
Hypernyms and hyponyms 21
Part-of-speech tagging 21
Named entity recognition 23
Chunking with rule-based grammars 24
Spell correction and “did you mean?” 24
Query expansion and query rewriting 24
Where this sits in modern retrieval 25
Intent Routing and Classification 27
The end-to-end query pipeline 27
Naive Bayes for text classification 27
Language detection as classification 28
Intent routing as classification 29
End-to-end walkthrough 30
Where this sits in modern retrieval 31
Summary 33
Method Comparison 33
Key Takeaways 33
Key Formulas 34
Self-Check Questions 34
Further Reading 35

3 - Advanced Text Processing

A user searches a bookshop's website for "car repair" and gets nothing useful. The catalog holds hundreds of matching titles, but their spines read "automobile repair", "auto repair", "Cars" (the Pixar film), or "car maintenance". The search index stores each of these as a separate token. "Car" and "automobile" never meet.

A different user asks "Who won the F1 race on the weekend?" and gets nothing useful either. The answer sits in a race-results page that reads "Verstappen took the win at Silverstone on Sunday, 6 July". That page shares almost no tokens with the query. "Who" does not appear in it; "weekend" does not appear either (the report says Sunday and gives a date); even "race" is absent because the article calls it a "Grand Prix". The user asked a question and expected a fact back. The retriever received a bag of five keywords and matched them against pages that also contain the word "weekend".

A third user types "Bücher von Goethe" and again gets unrelated results. Nothing in the pipeline noticed that "Bücher" is German for "books", that "Goethe" is a person's name, or that the sentence is a request for works by an author. All the structure the user provided was there in plain text and none of it was used.

The three failures come from the same source. Classical retrieval treats text as a bag of exact tokens: one surface form is one term and a query is a bag of query terms. That model works for keyword searches over clean text and breaks in two directions. On the document side, one concept has many surface forms and recall collapses. On the query side, natural-language input has structure, meaning, and intent that do not map to tokens in the answer at all, and the retriever ignores every layer of that information.

Advanced text processing addresses both. This chapter builds the classical toolbox that turns raw strings into features a retrieval and routing system can actually use: tokenization, normalization, sentence segmentation, stop words, stemming and lemmatization, phrase and compound detection, part-of-speech tagging, named entity recognition, and simple classifiers for language and intent. Together they close the recall gap on the document side and extract enough structure from queries that a retrieval system can route them, filter them, and rank the results.

The three failure examples do not all fully resolve in this chapter. The bookshop's "car repair" query and the German "Bücher von Goethe" query are handled end to end here. The F1 question is harder: the classical pipeline can turn it into a targeted news-search query with a date filter, but delivering "Verstappen won" as a two-word answer rather than a list of pages is the work of semantic search and retrieval-augmented generation in later chapters. The F1 case will follow us through the book.

What you'll learn

- Tokenize and normalize text for retrieval, including case folding, Unicode and accent handling, sentence segmentation, and language detection
- Compare stemming and lemmatization strategies and choose the right one for a given retrieval scenario
- Detect meaningful multi-word phrases and split compound words with pointwise mutual information, likelihood ratios, and log-frequency scoring
- Extract structure from free-text queries with part-of-speech tagging and named entity recognition
- Turn extracted features into decisions with a Naive Bayes classifier, applied to language detection and query intent routing

Prerequisites

- The retrieval architectures and text-processing basics introduced in 1 - Classical Text Retrieval
- The evaluation framework from 2 - Performance Evaluation, used to compare classifiers in the last section
- Basic probability and Bayes' rule, at the level covered in the ML foundations appendix

Tokenization and Normalization

The chapter opener showed a bookshop returning nothing for "car repair" because "automobile repair" is stored under a different token. That failure begins at the very first stage of the retrieval pipeline: how raw text is split into tokens and normalized before any matching happens. This section walks through that stage, starting with the whitespace-and-punctuation splitting from 1 - Classical Text Retrieval and ending with a rule-based language detector that decides which downstream pipeline to invoke.

The naive tokenizer and where it breaks

Chapter 1 split text at whitespace and punctuation boundaries. The simplest rendering of that approach is a one-line tokenizer:

```
def word_tokenize(text: str) -> list[str]:
    text = re.sub(r'[^\w\-]+', ' ', text)
    return [t for t in text.split(' ') if t]
```

It substitutes any non-word character with a space, then splits on whitespace. It works on well-behaved English prose. Give it a sentence that exercises real-world text and it fails in several distinct ways at once:

> I buy my parents' 10% of U.K. startup for $1.4 billion. Dr. Watson's cat called Mrs. Hersley and it was w.r.o.n.g., more to come …

Running the naive tokenizer produces:

```
['I', 'buy', 'my', 'parents', '10', 'of', 'U', 'K', 'startup',
 'for', '1', '4', 'billion', 'Dr', 'Watson', 's', 'cat', 'called',
 'Mrs', 'Hersley', 'and', 'it', 'was', 'w', 'r', 'o', 'n', 'g',
 'more', 'to', 'come']
```

Four systematic problems appear:

- **Possessives are handled inconsistently by shape**. `parents'` (trailing apostrophe only) becomes just `parents`: the apostrophe and the following space collapse into one space and the apostrophe leaves no trace. `Watson's` (apostrophe followed by `s`) becomes two tokens, `Watson` and a stray `s`, because the apostrophe becomes a space that splits the two letters apart. For retrieval, folding "Watson's" into "Watson" is often desirable (users search for the name, not the possessive), so the stray `s` is dropped in a subsequent cleanup step and both possessive forms end up equivalent. For syntactic analysis this is harmful, because the link between "Watson" and "cat" is broken and the shape-dependent behaviour is easy to overlook.
- **Numbers, currencies, and percentages fall apart**. `10%` becomes `10`, `$1.4 billion` becomes `1`, `4`, `billion`. Small integers survive; anything realistic does not.
- **Abbreviations disintegrate**. `U.K.`, `Dr.`, `Mrs.`, and the artificial `w.r.o.n.g.` all break at their internal periods. Searching for `U.K.` is impossible against this index.
- **Punctuation vanishes**. Fine for retrieval, but downstream sentence analysis loses its cues.

For a retrieval-only pipeline over English, this behaviour is not fatal: users rarely search for "U.K." with the period, and possessive stripping helps recall. But the distinctions are gone for good, and any task that needs sentence structure or numeric precision needs a better tokenizer up front.

Modern tokenization with nltk and spaCy

Both `nltk` and `spaCy` ship word tokenizers built from rules combined with abbreviation and exception lists learned from text corpora. Running each on the demonstration sentence:

```
import nltk
import spacy

sentence = ("I buy my parents' 10% of U.K. startup for $1.4 billion. "
```

```
            "Dr. Watson's cat called Mrs. Hersley and it was w.r.o.n.g., more to
come ...")

nltk.word_tokenize(sentence)
# ['I', 'buy', 'my', 'parents', "'", '10', '%', 'of', 'U.K.', 'startup', 'for',
#  '$', '1.4', 'billion', '.', 'Dr.', 'Watson', "'s", 'cat', 'called', 'Mrs.',
#  'Hersley', 'and', 'it', 'was', 'w.r.o.n.g.', ',', 'more', 'to', 'come', '...']

[t.text for t in spacy.load('en_core_web_sm')(sentence)]
# ['I', 'buy', 'my', 'parents', "'", '10', '%', 'of', 'U.K.', 'startup', 'for',
#  '$', '1.4', 'billion', '.', 'Dr.', 'Watson', "'s", 'cat', 'called', 'Mrs.',
#  'Hersley', 'and', 'it', 'was', 'w.r.o.n.g', '.', ',', 'more', 'to', 'come', '...']
```

The two outputs agree on every token except one: nltk treats `w.r.o.n.g.` as a single abbreviation, spaCy splits the trailing period as sentence-terminating punctuation. On real text the two libraries usually agree; artificial or novel abbreviations are where they diverge.

Compared with the naive regex, the trained tokenizers fix several of the earlier failures but not all of them, and the two possessive shapes still behave differently:

- **Possessives**. `parents'` produces `parents` and a bare `'`. `Watson's` produces `Watson` and `'s`. Both tokenizers keep the apostrophe visible; only the `'s` form carries a marker that distinguishes it from any other apostrophe. Retrieval pipelines usually drop both apostrophe tokens after this step; syntactic pipelines keep them because they carry grammatical information.
- **Numbers**. Decimals stay together (`1.4`), which fixes the naive regex's worst behaviour. But the currency and percent symbols split off as their own tokens: `$1.4` becomes `['$', '1.4']` and `10%` becomes `['10', '%']`. A downstream step that wants "$1.4 billion" as one money entity needs to recombine, which is the job of the named entity recognizer covered later in the chapter.
- **Abbreviations**. `U.K.`, `Dr.`, and `Mrs.` are kept whole with their trailing periods, because both libraries carry curated abbreviation lists. `w.r.o.n.g.` is the divergent case: nltk keeps it whole, spaCy splits its final period. Real abbreviations rarely differ between the two libraries.
- **Punctuation**. `.`, `,`, and `...` are preserved as their own tokens. Retrieval pipelines filter them out; syntactic pipelines keep them.

For retrieval, we still want a leaner token list than the tokenizer alone provides. The standard cleanup after tokenization is:

1. Drop single-letter tokens and pure punctuation tokens.
2. Drop numbers and currency tokens unless the collection is numeric (a scientific-paper index would keep them).
3. Drop possessive tokens (`"'s"`) once the possessive information has been used.

The chapter's demo notebook composes these three cleanup steps on top of a regex tokenizer and compares the intermediate outputs side by side.

Case, Unicode, and accents

A tokenizer separates words. Normalization decides which surface forms of the same word collapse to the same token. Consider three variants of one city name:

`Zurich`, `Zürich`, `ZURICH`

They refer to the same city. To match a query for `Zurich` against a document that spells the city `Zürich`, the index has to normalize both spellings to a shared form before storing them. Three transforms handle almost all cases in European text:

- **Case folding** applies `text.lower()`, or the locale-aware equivalent for Turkish `İ`/`ı`. The three variants collapse to `zurich`, `zürich`, `zurich`.

- **Unicode normalization** applies `unicodedata.normalize('NFKC', text)` to canonicalise characters that have multiple valid encodings, such as the ligature ﬁ (one codepoint) unfolding to `fi` (two ASCII letters).
- **Accent folding** strips diacritics after normalization, mapping `ü -> u`, `é -> e`, `ß -> ss`. The library `unidecode` does this for arbitrary Unicode; `nltk`'s stemmers apply narrower per-language rules.

After all three, the three variants above become `zurich`. Users can type any spelling and match any document.

> **Accent folding is not free**
>
> Aggressive accent folding trades recall for precision. In German, "schon" (already) and "schön" (beautiful) have different meanings; folding both to "schon" makes them collide in the index. In French, "où" (where) and "ou" (or) become the same token. Whether to fold depends on the retrieval scenario: web search over cross-language content typically folds, while a legal-document index over a single language does not.

In practice, modern text-processing stacks apply Unicode normalization by default because it has no downside, case-fold when the retrieval scenario is case-insensitive (most web search), and fold accents only when the user population cannot easily type them.

Sentence segmentation

Some tasks need whole sentences as input, not tokens. Part-of-speech taggers work sentence by sentence because word ambiguity resolves in context. RAG systems (see the retrieval-augmented generation chapter) build larger chunks by grouping consecutive sentences until they hit a size limit. Both need a reliable answer to the question: where does one sentence end and the next begin?

Splitting on `.` is the naive approach and fails on the same abbreviations we saw earlier. In "Dr. Watson's cat called Mrs. Hersley.", the periods after `Dr` and `Mrs` are not sentence boundaries; only the trailing period is. Multi-line quoted dialogue, ellipses, and decimal numbers add more edge cases.

Two libraries handle this well:

- **Punkt** (NLTK) is an unsupervised sentence segmenter that learns abbreviations and sentence-start patterns from a training corpus. NLTK ships pre-trained models for English and several other languages. It is fast and accurate on newspaper-style prose; it struggles on narrative dialogue where punctuation appears both inside and outside quotation marks.
- **spaCy** segments sentences as a side effect of its full linguistic pipeline. It is heavier than Punkt but handles complex sentence structures more consistently and supports many languages out of the box.

```
from nltk.tokenize import sent_tokenize
sent_tokenize("Dr. Watson's cat called Mrs. Hersley. She was Egyptian.")
# ["Dr. Watson's cat called Mrs. Hersley.", "She was Egyptian."]
```

Sentence segmentation is one of the few pieces of a classical pipeline that stayed useful into the LLM era. Every RAG system built on top of embeddings still needs to decide where its chunks begin and end, and sentence boundaries are the most reliable low-cost signal for that decision.

Word boundaries are not always spaces

Not every writing system uses spaces to separate words, and not every "word" in a modern document is meant to be a single token. Both cases break the "split on whitespace" assumption at the root.

Scriptio continua languages, including Chinese, Japanese, Thai, and classical Greek, do not use spaces at all. A single sentence is a continuous string of characters:

```
莎拉波娃现在居住在美国东南部的佛罗里达。
```

```
莎拉波娃    现在    居住   在    美国    东南部       的      佛罗里达
Sharapova  now    lives  in   US    southeast    in      Florida
```

Code identifiers pack multiple words into a single token by convention: `QueryParser`, `assertEquals`, `word_tokenize`. A code-search index that stores only whole identifiers cannot answer a query for `parser`.

Spoken-language transcripts come from a continuous phoneme stream. Speakers do not pause between words; the pauses that exist mark breath, hesitation, or emphasis, and rarely align with word boundaries. Even the pause at a sentence end is unreliable in fluent speech. The segmenter has to infer word boundaries from the phoneme sequence itself, aided by a pronunciation dictionary and a language model, and non-native pronunciation or unclear articulation shifts where those boundaries land.

There are two ways to break these continuous streams into useful tokens:

- **Dictionary lookup with a probabilistic tie-breaker**. The dictionary lists every candidate word starting at the current position, and a trained model (historically an HMM, today usually a neural network) picks the most likely segmentation. This works well when the vocabulary is closed and the domain is stable. It breaks on names, brand terms, and loanwords.
- **Sub-word tokens used directly for retrieval**. Instead of reconstructing words, we index overlapping character n-grams. For languages that *do* have whitespace, a common choice is trigrams with a `#` boundary marker indicating that a trigram starts at a word boundary (Cavnar & Trenkle 1994):

```
This course teaches multimedia retrieval.
#Thi his #cou our urs rse #tea eac ach che hes #mul ult lti tim ime med edi dia ...
```

For scriptio-continua text where no spaces exist, the `#` markers are simply omitted and the entire character stream is shingled into overlapping trigrams. Either way, queries are encoded identically:

```
Q = "teach multtimedia"
    #tea eac ach #mul ult ltt tti tim ime med edi dia
```

Even with a different inflected form (“teaches” versus “teach”) and a typo (“multtimedia”), 10 of the 12 query trigrams match trigrams from the document. A retrieval model that supports partial matching with a proximity bonus finds the passage without any explicit stemming or spell-check.

This workaround for unsegmentable text turned out to be a foundational idea. Byte Pair Encoding (BPE) and WordPiece take the same principle (match parts of words when whole-word matching fails) and learn the token vocabulary from a corpus instead of fixing it to trigrams. Both power today’s large language models, and the semantic-search chapter covers them in detail.

Language detection

Both indexing documents and understanding queries depend on knowing what language the input is in. A German document benefits from Snowball’s German rules and the German stop-word list; running it through the English pipeline strips accents but leaves compounds and inflections untouched. A query like “Bücher von Goethe” from the chapter opener needs to be recognized as German before the retrieval system can restrict results to German-language books or apply German-specific normalization.

For long documents, a small number of hand-crafted rules is enough:

- **Alphabet diversity** looks at which scripts appear. Latin, Cyrillic, Greek, Arabic, Devanagari, Thai, and CJK (Chinese, Japanese, Korean) are easy to tell apart even from a single character.
- **Character diversity** narrows down within a script. ä, ö, ü are strong signals for German (and Turkish, Estonian, Finnish); ç for French, Portuguese, or Turkish; ñ for Spanish; ß almost exclusively for German.
- **Stop-word counts** track how many of the top hundred function words of each candidate language appear. English “the”, “of”, “and”; German “der”, “die”, “und”, “ist”; French “le”, “la”, “de”, “et”. Whichever language’s stop-word list matches most terms wins.
- **Vocabulary counts** extend the stop-word idea to a longer list of frequent content words.

For document-side indexing this is usually enough: a paragraph or a page provides many opportunities for the rules to fire, and the winning language emerges quickly. Once the language is known, the pipeline branches: a German document goes to the German Snowball stemmer, an English document to the English WordNet lemmatizer, and so on.

Short queries break the rules, though. “Mein computer” mixes German with an English loanword; “pain” is a French word for bread and an English word for suffering. Neither has enough evidence for the rules to fire reliably. That regime needs a statistical classifier over character n-grams, developed with Naive Bayes in Intent Routing and Classification.

Where this sits in modern retrieval

Tokenization and normalization are the least glamorous part of the retrieval stack and still the largest source of quality issues in production. Modern lexical retrievers like Lucene, Solr, and Elasticsearch expose the whole pipeline as a chain of configurable analyzers: a tokenizer, a lowercase filter, a Unicode-normalization filter, a stop-word filter, a stemmer, sometimes a compound splitter. Dense retrieval sidesteps most of these choices by learning them implicitly in the embedding model, but hybrid stacks (BM25 combined with dense retrieval) still need the lexical side, and the lexical side still starts with these steps. Getting them wrong at ingestion time is expensive to fix later because the index has to be rebuilt.

Hands-on: Tokenization and Normalization

Run the naive regex tokenizer, the nltk word tokenizer, and the spaCy tokenizer on the “Dr. Watson” sentence. Normalize the results with case folding and accent stripping, and observe the effect on a small BM25 index.

Open notebook ->

Includes pre-run results; you can read through or download and experiment.

The next section takes tokens as given and looks at the second half of the recall gap: collapsing “car”, “cars”, “carrying”, and “carried” so a document mentioning any of them matches a query for any other.

Stemming and Lemmatization

The bookshop catalog stores documents word by word. A user searches “car repair” and misses titles like “cars”, “repairing cars”, and “repaired vehicles” because each surface form is a distinct token in the index. Tokenization from the previous section fixed splitting; it did not fix the fact that one root word has many written shapes. This section closes that gap. First it decides which tokens are worth indexing at all, then it collapses inflected surface forms of the same word onto a common stem or lemma.

Stop words: what to keep out

Half a dozen very frequent words dominate any English text. “The”, “of”, “and”, “to”, “is” together account for roughly a quarter of tokens in typical prose. We already saw in From Text to Searchable Vectors that these high-frequency terms appear in over 60% of documents yet carry almost no content signal. A **stop-word list** is the standard shortcut: a fixed set of high-frequency function words to discard before indexing and querying. English stop-word lists usually contain 100-200 words. The filtering logic is straightforward:

```
STOPWORDS = {'a', 'an', 'the', 'and', 'or', 'in', 'of', 'to', ...}

def remove_stopwords(tokens: list[str], stopwords: set = None) -> list[str]:
    sw = stopwords if stopwords is not None else STOPWORDS
    return [t for t in tokens if t not in sw]
```

Every major language has its own list. `nltk.corpus.stopwords.words('german')` returns 232 German function words; French, Spanish, Italian, and the other main European languages are provided by both `nltk` and `spaCy`.

Dropping stop words works well most of the time and fails visibly the rest of the time. The cost is a small class of queries where the stop word is content-bearing:

- A search for **Stephen King’s “It”** finds nothing if the tokenizer drops “it” as a stop word. The novel’s title is the stop word.
- A search for **“IT security”** loses “IT” (information technology) for the same reason. The remaining “security” retrieves every document about locks, guards, and stock exchanges.
- **“To be or not to be”** consists entirely of stop words. Every word disappears.
- **“The Who”** (the band) has both tokens on typical stop lists.

BM25 already handles high-frequency words gracefully through two mechanisms that earlier bag-of-words models lacked: inverse document frequency drives the weight of corpus-wide common terms toward zero, and term-frequency saturation prevents any single term from dominating a document’s score no matter how often it repeats. Together these make aggressive stop-word removal less necessary. The modern default is to keep stop words in the index (storage is cheap) but skip them when generating phrase collocations, computing term-based similarities, or building compact features for a classifier.

> **Never strip stop words from a phrase query**
>
> Dropping stop words before phrase matching turns “The Who” into an empty query and “not guilty” into “guilty”. If the retrieval scenario supports phrase or entity queries, the stop-word filter must be aware of the phrase context or must not run on those tokens at all.

Stemming: rule-based reduction

Once stop words are out, we still have “car”, “cars”, “carrying”, “carried”, “carrier”. They all point to the same concept, but a bag-of-words index sees five distinct tokens. **Stemming** applies a set of ordered rules that strip suffixes off inflected forms so that variants of one root end up as the same token.

The three widely-used English rule-based stemmers are Porter, Lancaster, and Snowball. All three take a token, apply an ordered sequence of suffix-stripping rules, and return a stem. The stem does not need to be a real English word; it only needs to be the same for all variants of the same lemma.

The Porter algorithm

The Porter algorithm is the classical English stemmer, published by Martin Porter in 1980. Chapter 1 introduced it as a black box; here we look at the internal structure.

Porter defines a **vowel** as A, E, I, O, U, and additionally Y when the preceding character is a consonant (so "Y" is a vowel in "rhythm" but a consonant in "yellow"). All other characters are consonants. Let `C` be a sequence of consonants and `V` a sequence of vowels. Every English word can be written in the form:

$$[C](VC)^m[V] \tag{3.1}$$

where the exponent m is the **measure** of the word. `TREE` has $m = 0$ (consonant sequence `TR` followed by vowel sequence `EE`, no `VC` pair in between), `TROUBLE` has $m = 1$, and `TROUBLES` has $m = 2$. Porter uses m to prevent over-stemming: rules that would strip "ATE" from "OPERATE" are gated behind $m \geq 1$ so they do not also strip the "ATE" from "MATE".

The algorithm applies five steps in sequence, containing roughly 60 suffix-replacement rules in total. The entire implementation fits in about 400 lines of Python (or the equivalent in Java/C). Each step targets a different layer of English morphology:

Step	Goal	Example rules	Example
1a	Strip plurals	SSES arrow.r SS, IES arrow.r I, S arrow.r ∅	caresses arrow.r caress, ponies arrow.r poni, cats arrow.r cat
1b	Strip past tense / progressive	($m > 0$) EED arrow.r EE, (v) ED arrow.r ∅, (v) ING arrow.r ∅	feed arrow.r feed, plastered arrow.r plaster, motoring arrow.r motor
2	Simplify derivational suffixes	($m > 0$) ATIONAL arrow.r ATE, ($m > 0$) IZER arrow.r IZE	relational arrow.r relate, digitizer arrow.r digitize
3	Continue simplification	($m > 0$) ICATE arrow.r IC, ($m > 0$) ALIZE arrow.r AL	triplicate arrow.r triplic, formalize arrow.r formal
4	Remove residual suffixes	($m > 1$) ION arrow.r ∅, ($m > 1$) ISM arrow.r ∅	adoption arrow.r adopt, platonism arrow.r platon
5	Clean up trailing letters	($m > 1$) E arrow.r ∅, double consonant arrow.r single	rate arrow.r rate, controll arrow.r control

Within each step, rules are tried in order and only the first match fires. The conditions in parentheses prevent rules from applying to stems that are too short: ($m > 0$) requires at least one vowel-consonant pair in the remaining stem, and (v) requires at least one vowel anywhere. This is what keeps Step 1b from stripping "ED" off a two-letter word.

The full rule set stems on the order of 95% of English text correctly. Porter's original paper (Porter, 1980) lists every rule; the `nltk.PorterStemmer` implementation is faithful to that specification.

```
import nltk
porter = nltk.PorterStemmer()

for w in ['car', 'cars', 'carrying', 'carried', 'carrier']:
    print(w, '->', porter.stem(w))

# car       -> car
# cars      -> car
# carrying  -> carri
# carried   -> carri
# carrier   -> carrier
```

Note that "carrying" and "carried" collapse to `carri` (not to `carry`), and "carrier" is left intact. The stems are not linguistically pretty, but they map three variants of the verb "carry" onto the same token, which is what the retrieval index needs.

Lancaster and Snowball

Lancaster (Paice, 1990) is a more aggressive stemmer for English. It applies iterative rule-based suffix removal until no rule fires, producing shorter stems than Porter. Speed is its main advantage: on average it does fewer rule scans per word. Its main risk is collisions: unrelated words can reduce to the same short stem.

```
lancaster = nltk.LancasterStemmer()
for w in ['one', 'only', 'organization', 'organize']:
    print(w, '->', lancaster.stem(w))

# one          -> on
# only         -> on
# organization -> org
# organize     -> org
```

"one" and "only" both stem to "on", so a document about "the only book on Rome" collides with one about "book number one". For a scenario that favors recall the collisions are usually acceptable; for a name search or a precise query they are a problem.

> **Lancaster over-stems short words**
>
> Lancaster reduces short words to two- or three-letter stems that collide across unrelated meanings ("one" and "only" both become "on"). Use Lancaster when speed dominates and the collection is large enough that a few token collisions do not affect ranking. Prefer Porter or Snowball on short-document collections where each collision matters.

Snowball is Martin Porter's own multi-language framework, published later, that generalizes the Porter design. It supplies stemmers for around 20 European languages including English, German, French, Spanish, Italian, Dutch, and Russian. On English, Snowball is essentially a revised Porter with a handful of small corrections; on other languages, it is often the only rule-based option available.

```
snowball_en = nltk.SnowballStemmer("english")
snowball_de = nltk.SnowballStemmer("german")

snowball_de.stem("Wagen"), snowball_de.stem("Wägen")   # ('wag', 'wag')
snowball_de.stem("Reparatur"), snowball_de.stem("reparieren")  # ('reparatur',
'repar')
```

The German stemmer folds umlauts (`Wägen -> Wagen -> wag`) and reduces "reparieren" to `repar`, which does not match `reparatur`. Rule-based stemmers approximate the linguistic stem but do not compute it: for German, the noun "Reparatur" and the verb "reparieren" share a root but have different stems under any pure suffix-stripping rule set.

Lemmatization: dictionary-based reduction

Rule-based stemming is fast and needs no external data, but it produces linguistically incorrect stems and cannot handle strong inflections. English "go" and "went" share no letters at all, so no suffix rule can map them together. **Lemmatization** takes a different approach: look the word up in a dictionary and return its recorded base form.

The three components of a dictionary-based lemmatizer are:

1. A **rule set** for regular inflections (`-s`, `-ed`, `-ing`, plurals with `-ies`).
2. An **exception list** for irregular forms: `went -> go`, `ate -> eat`, `mice -> mouse`, `children -> child`.
3. A **dictionary** of all base forms in the language, used to check whether a candidate form is valid.

The algorithm proceeds in order:

1. Take the current token and its part-of-speech (POS) tag (verb, noun, adjective, ...). POS tagging is covered later in this chapter; what matters here is that "meeting" as a noun lemmatizes to `meeting`, while "meeting" as a verb lemmatizes to `meet`.
2. If the token is in the dictionary as-is, it is already a base form. Return it.
3. If the token is in the exception list for its POS, return the base form listed there.
4. Apply the regular-inflection rules for its POS, checking the dictionary after each rule. Return the first match.
5. If no rule produces a dictionary word, return the token unchanged. This handles names, misspellings, and loanwords.

`WordNetLemmatizer` in `nltk` and the built-in lemmatizer in `spaCy` are the two standard implementations. Their exception lists cover several thousand irregular forms each.

```
wn = nltk.WordNetLemmatizer()
wn.lemmatize('carried', 'v')    # 'carry'
wn.lemmatize('carrier', 'n')    # 'carrier'
wn.lemmatize('mice', 'n')       # 'mouse'
wn.lemmatize('went', 'v')       # 'go'
wn.lemmatize('better', 'a')     # 'good'
```

Note the last case: "better" lemmatizes to "good" because the exception list encodes the adjective's comparative form. No rule-based stemmer can do this.

`spaCy` runs a full linguistic pipeline that assigns POS tags automatically, so the lemmatizer is applied without the caller having to pass a tag:

```
nlp = spacy.load('en_core_web_sm')
for token in nlp("She carried the mice yesterday and went home"):
    print(token.text, '->', token.lemma_)

# She       -> she
# carried   -> carry
# the       -> the
# mice      -> mouse
# yesterday-> yesterday
# and       -> and
# went      -> go
# home      -> home
```

For retrieval, the practical difference between rule-based stemming and dictionary lemmatization is that a lemmatizer gives you the linguistic base form and a stemmer gives you an arbitrary reduction that just happens to be consistent within the class. Both work for matching a query to a document. Lemmatization is heavier because it needs POS tagging plus a dictionary lookup; stemming is a handful of string operations.

Multi-lingual comparison

Every stemmer and lemmatizer described above assumes one language. Running an English stemmer on French text strips accents and collapses "les" to "l" without doing anything useful. The right choice depends on the language of the input, which is what the language detector from the previous section is for.

Snowball is the go-to rule-based option outside English because it covers 20 languages under a single framework. spaCy is the go-to lemmatizer for the same set. On the same French sentence, the two diverge on inflected verb forms:

```
Input:      "Nous avons aperçu les châteaux"

Snowball:   ['nous', 'avon',  'aperçu',     'le', 'château']
spaCy:      ['nous', 'avoir', 'apercevoir', 'le', 'château']
```

Snowball preserves accents in the French output (visible on `aperçu` and `château`) but it does not recognize inflected verb forms: `avons` becomes `avon` by simple suffix stripping rather than mapping to `avoir`, and

`aperçu` stays unchanged rather than lemmatizing to the infinitive `apercevoir`. spaCy's French lemmatizer handles both because it has the verb tables. Both correctly reduce the plural `châteaux` to `château` while keeping the circumflex. On entity-heavy queries the difference is small; on prose retrieval where verbs carry the topic, it moves the recall needle.

In German, nouns inflect for case, gender, and number, and German also forms diminutives with "-chen" and "-lein". The two tools handle these differently:

```
Input:      "die Häuser und ihre Häuschen"

Snowball:   ['die', 'haus', 'und', 'ihr', 'hausch']
spaCy:      ['der', 'Haus', 'und', 'ihr', 'Häuschen']
```

Both reduce the plural "Häuser" to the singular ("haus" for Snowball, which also lowercases; "Haus" for spaCy). They part ways on the diminutive "Häuschen": Snowball over-stems it to "hausch", a token that matches neither "Haus" nor any real word, while spaCy keeps "Häuschen" as its own lemma. For queries against German text, spaCy's dictionary lemmatizer avoids these spurious stems, though the small models are weak on strong verbs and do not always map inflected verb forms to a common infinitive.

Neither approach handles compound splitting, which is the concern of the next section: "Wolkenkratzer" (skyscraper) stays as a single token under both stemmers.

Where this sits in modern retrieval

Rule-based stemming and dictionary lemmatization are still the default text-processing configuration for lexical retrievers. Lucene, Solr, and Elasticsearch ship analyzers for every major language that combine tokenization, stop-word filtering, and stemming or lemmatization in the same order this section presented them. Stemming is the option chosen when performance matters or when no dictionary is available; lemmatization is preferred when accuracy on strongly-inflected languages or on strong-verb English matters more than throughput.

Dense retrieval learns these normalizations implicitly during embedding training. A well-trained embedding model puts "carried", "carrying", and "carrier" near each other in vector space without an explicit stemming step. This does not make the lexical stack obsolete: hybrid retrieval combines BM25 (which needs stemming or lemmatization to keep recall high) with dense retrieval (which does not), and the two together consistently outperform either alone. The classical stemmers of this section are still on the query path of every serious production search stack in 2025.

Hands-on: Stemming and Lemmatization

Compare Porter, Lancaster, Snowball, WordNet, and spaCy on English, German, and French text. See how each stemmer treats strong verbs, plurals, and compounds.

Open notebook ->

Includes pre-run results; you can read through or download and experiment.

The next section takes stemmed and lemmatized tokens as given and looks at what happens when meaning crosses word boundaries: "New York" is not "New" plus "York", and "Wolkenkratzer" is not "Wolke" plus "kratzer".

Phrases and Compounds

Tokenization decides what a token is; stemming and lemmatization decide which tokens are equivalent. Both operate at the level of one word. This section handles the two cases where a one-word view of meaning breaks down. Phrases pack multiple words into one concept: “New York” is not “New” plus “York”, and a query for “New” retrieves the wrong documents. Compounds do the opposite: “Bücherregal” (bookcase) is one German token built from “Bücher” (books) and “Regal” (shelf). A shopper on an e-commerce site who searches for “Regal” will not see the bookcase listings until we split the compound into its parts. Both cases need explicit handling before the retrieval model sees the tokens.

Bi-grams, tri-grams, phrases

The naive bag-of-words model treats a document as a multiset of independent tokens. “New York City”, “Salt Lake City”, and “prime minister” become bags of three or two unrelated tokens. Searching for “New York” as two separate terms still retrieves every document that mentions the city, so recall is not the problem. Precision is: the query also matches documents where “new” and “York” occur unrelated (“a new book from York University”), and a plain bag-of-words index has no way to tell those apart from documents where the two words sit side by side. Without a proximity or phrase mechanism, the genuine “New York” documents cannot be ranked above the incidental co-occurrences. Indexing “New York” as a single token `new_york` restores precision and gives the ranker a clean signal.

This generalises to n-grams: a bi-gram joins two adjacent words, a tri-gram three, and so on. Indexing the bi-grams and tri-grams of “the city of New York City” produces

```
the, city, of, new_york_city, new_york, york_city, city
```

with _ marking a phrase token. We keep the original single-word tokens and emit every overlapping n-gram rather than only the longest one. This is deliberate: a query for “New York” must still match even though this document only ever contains the longer “New York City”, and a query for the single word “city” must still match as well. Keeping all of them costs some index space but avoids missing any of these queries. A phrase query for “New York” now hits `new_york` directly, without proximity constraints. The question that follows is which n-grams are worth indexing. Adding every bi-gram in the corpus is prohibitive (it turns a 100k-token vocabulary into a 10-billion-pair vocabulary) and most of them are noise: “it is”, “of the”, “and to”. We need a scoring function that selects the small subset of bi-grams that mean something the individual tokens do not.

Naive frequency and the stop-word problem

The obvious approach is to count how often each bi-gram appears in the corpus and keep the most frequent ones. On any English text this immediately breaks:

```
of the      12,847
in the       9,203
to the       7,451
and the      6,988
for the      5,102
...
```

The top of the list is dominated by pairs of stop words. Filtering these out helps but does not solve the problem: even after stripping stop-word pairs, high-frequency co-occurrences of common words (“said Holmes”, “young man”, “could see”) still bubble to the top and squeeze out the phrases we actually want. Frequency alone cannot tell us whether two words appear together because they mean something together or because they are both common.

Two better scoring functions capture that distinction: pointwise mutual information (PMI) and the likelihood ratio (LHR).

Pointwise mutual information

The intuition behind **pointwise mutual information** is straightforward. Two words that mean something together should co-occur more often than their individual frequencies would suggest under independence. In *A Study in Scarlet*, a corpus of about 43,200 tokens, "said" appears 207 times and "Holmes" 98 times. If the two were independent, we would expect the bi-gram "said Holmes" to occur about $207 \cdot 98/N \approx 0.5$ times. It actually occurs 12 times, roughly 25 times more than chance. That is a real signal, but a modest one: "said" precedes dozens of different names throughout the book.

Compare "Sherlock Holmes". "Sherlock" appears 52 times, and every one of those 52 occurrences is followed by "Holmes". Independence would predict about $52 \cdot 98/N \approx 0.1$ co-occurrences; the observed count of 52 is more than 400 times higher. "Sherlock" is effectively bound to "Holmes".

Key Formula: Pointwise Mutual Information

$$\operatorname{pmi}(t_1, t_2) = \log_2 \frac{p(t_1, t_2)}{p(t_1) \cdot p(t_2)} = \log_2 \frac{N \cdot \operatorname{tf}(t_1, t_2)}{\operatorname{tf}(t_1) \cdot \operatorname{tf}(t_2)} \tag{3.2}$$

Here $\operatorname{tf}(t)$ is the number of times term t occurs in the corpus, and $\operatorname{tf}(t_1, t_2)$ is the co-occurrence count: how often the bi-gram appears, that is, how often t_2 immediately follows t_1. Dividing each count by the corpus size N turns it into a probability: $p(t) = \operatorname{tf}(t)/N$ for a single term and $p(t_1, t_2) = \operatorname{tf}(t_1, t_2)/N$ for the pair. PMI is then the log-ratio of the observed joint probability to the joint probability that independence would predict; it is high when two tokens almost always appear together and rarely apart.

Two things help read the formula. First, $\log_2(N)$ is the same constant for every bi-gram, so ranking by PMI is the same as ranking by $\log_2 \operatorname{tf}(t_1, t_2) - \log_2 \operatorname{tf}(t_1) - \log_2 \operatorname{tf}(t_2)$. Second, look at what maximizes the score. PMI is largest when all three counts equal 1: a word that occurs once, sitting next to another word that occurs once, giving $\operatorname{pmi} = \log_2 N$. Even when the two words and their pair all occur n times together and never apart, the score is $\log_2(N/n) = \log_2 N - \log_2 n$, which only falls as n grows. So PMI rewards rarity, and this is the mirror image of the stop-word problem: instead of frequent function words dominating, two rare words that happen to land side by side by chance float to the top. In *A Study in Scarlet*, 199 different bi-grams reach the maximum score of $\log_2 N \approx 15.4$, every one of them a pair of words that each occur exactly once and only next to each other ("admired treated", "airy cheerfully", "ambitious title"). All of them outrank "Sherlock Holmes". A single chance adjacency is no evidence of a real collocation; there is simply too little frequency to be confident the two words prefer each other. The cure is a minimum-frequency filter: require a bi-gram to occur at least, say, three times before scoring it at all, which drops the low-evidence rare pairs.

Example: PMI on the Sherlock Holmes corpus

Tokens are lowercased and reduced to alphabetic-only words, as in the normalization step of the previous section. Ranked by PMI with a minimum bi-gram frequency of 3 and a stop-word filter, the top of the list for *A Study in Scarlet* (about 43,200 tokens, 115 bi-grams surviving the filter) looks like this:

Rank	Bi-gram	$\operatorname{tf}(t_1, t_2)$	$\operatorname{tf}(t_1)$	$\operatorname{tf}(t_2)$	PMI
1	audley court	3	3	4	13.40
2	torquay terrace	3	3	4	13.40
3	avenging angels	4	4	4	13.40
4	lauriston gardens	6	6	6	12.81
8	salt lake	10	11	10	11.94
11	scotland yard	6	8	9	11.81
…	…				
45	sherlock holmes	52	52	98	8.78

Even after the frequency filter, PMI's rare-word bias persists. The top three are tied at 13.40 because each reduces to $\log_2(N/4)$: "audley court" and "torquay terrace" pair a word occurring 3 times with one occurring 4 times, always together, and "avenging angels" pairs two words occurring 4 times each. All are rare pairs whose parts appear almost only together. "Sherlock Holmes", the defining phrase of the book, ranks only 45th of 115: because "Holmes" occurs 98 times, the larger denominator drags its PMI down to 8.78. The filter removed the tf=1 noise but did not fix the underlying preference for rarity.

Likelihood ratio

The likelihood ratio test attacks the same problem from a hypothesis-testing angle rather than an information-theoretic one. It asks: how much better is the "these words are dependent" hypothesis than the "these words are independent" hypothesis, given the observed counts?

Let $\mathrm{tf}_1 = \mathrm{tf}(t_1)$, $\mathrm{tf}_2 = \mathrm{tf}(t_2)$, and $\mathrm{tf}_{12} = \mathrm{tf}(t_1, t_2)$. Two hypotheses:

- H_1 (independence): the probability that t_2 follows any word is a single value p, whether the previous word is t_1 or not. The maximum-likelihood estimate is $p = \mathrm{tf}_2/N$.
- H_2 (dependence): the probability that t_2 follows t_1 is $p_1 = \mathrm{tf}_{12}/\mathrm{tf}_1$, and the probability that t_2 follows any word other than t_1 is $p_2 = (\mathrm{tf}_2 - \mathrm{tf}_{12})/(N - \mathrm{tf}_1)$, with $p_1 \neq p_2$.

Assuming a binomial distribution for the "next word is t_2" events, the likelihood of each hypothesis is:

$$L(H_1) = b(\mathrm{tf}_{12}; \mathrm{tf}_1, p) \cdot b(\mathrm{tf}_2 - \mathrm{tf}_{12}; N - \mathrm{tf}_1, p) \tag{3.3}$$

$$L(H_2) = b(\mathrm{tf}_{12}; \mathrm{tf}_1, p_1) \cdot b(\mathrm{tf}_2 - \mathrm{tf}_{12}; N - \mathrm{tf}_1, p_2) \tag{3.4}$$

with $b(k; n, x) = \texttt{binom}\ nkx^k(1-x)^{n-k}$.

Key Formula: Log Likelihood Ratio

$$\log \lambda = \log \frac{L(H_1)}{L(H_2)} \tag{3.5}$$

The log-ratio of the likelihood under independence to the likelihood under dependence. Values near zero mean the two hypotheses fit the data equally well; strongly negative values mean dependence fits much better and the bi-gram is genuine. Bi-grams are usually ranked by $-2\log\lambda$, which under H_1 follows a chi-squared distribution with one degree of freedom.

LHR is more robust than PMI on sparse data because it directly compares two probabilistic models rather than comparing observed to expected counts. It does not have PMI's rare-word bias, so infrequent bi-grams score less aggressively. The trade-off is that LHR ranks frequent grammatical pairs highly, because those pairs really are non-independent even though they are useless as phrases. The bi-gram "I am" is the clearest example: "am" occurs 41 times and 39 of them follow "I", so the dependence is overwhelming, yet "I am" is worthless as an index phrase. This is why LHR still needs a stop-word filter: without one, "of the", "to be", "had been", and "I am" crowd the top of the ranking alongside the genuine names.

Example: LHR on the Sherlock Holmes corpus

Ranked by $-2\log\lambda$ on the same corpus, with the same minimum-frequency and stop-word filter as the PMI table:

Rank	Bi-gram	$\mathrm{tf}(t_1, t_2)$	$\mathrm{tf}(t_1)$	$\mathrm{tf}(t_2)$	$-2\log\lambda$
1	sherlock holmes	52	52	98	668
2	jefferson hope	34	37	56	458
3	john ferrier	29	39	62	352
4	brixton road	13	15	27	188

5	salt lake	10	11	10	181
9	scotland yard	6	8	9	98
11	baker street	6	6	29	89

With the stop-word filter in place, LHR puts the main characters and places of the novel on top: “Sherlock Holmes”, “Jefferson Hope”, “John Ferrier”, “Brixton Road”. Without the filter, the grammatical pairs discussed above (“of the”, “to be”, “I am”) would sit among them. The filter removes that whole class in one step.

PMI or LHR: which to use

The two measures rank the same corpus differently because they measure different things. PMI measures the *strength* of association: how much more often two words appear together than chance predicts, regardless of how often that is. LHR measures the *evidence* for association: how confidently the data rules out independence, which grows with frequency. “Sherlock Holmes” makes the contrast concrete: it ranks 45th of 115 under PMI, held down because “Holmes” is common, but first under LHR, which rewards the sheer weight of 52 co-occurrences.

Because they optimise different quantities, the two fail in opposite directions, and a genuine phrase can score well on one and poorly on the other:

- **Rare but tight** phrases occur a handful of times, always as a unit. They score high on PMI and low on LHR. In our results these are PMI’s top rows: “Lauriston Gardens” (the address of the murder), “Audley Court”, “chemical laboratory”. LHR buries them below its frequency-driven head, so PMI is what surfaces them.
- **Frequent but loose** pairs such as “young man” or “could see” occur often but are not real phrases. They score moderately on LHR but low on PMI, which correctly discounts them because the individual words are common.
- **Frequent and tight** phrases such as “Sherlock Holmes” or “Jefferson Hope” score high on both and need no adjudication.

No single measure is uniformly best. Some practical guidance:

- **For a phrase vocabulary that maximises recall** (finding as many genuine phrases as possible), take the top-k from both measures and union them: LHR contributes the well-attested head, PMI the rare-but-specific tail of named entities and technical terms that users actually search for. A union only adds candidates, so plan a downstream prune, whether a human review pass or a secondary filter, to drop the frequent-but-loose pairs that LHR lets through.
- **For a compact, high-precision list**, use LHR alone with the stop-word filter and a frequency floor. Its preference for well-attested pairs is an asset here, and you accept missing some rare phrases.
- **The filters matter more than the choice of measure.** A minimum-frequency floor removes PMI’s chance pairs and a stop-word filter removes LHR’s grammatical pairs; without both, neither measure produces a usable list. Scale the frequency floor to the corpus: 3 works for a single novel, but a large collection needs a higher floor to keep the vocabulary manageable.
- **An alternative to the union** is to use one measure to rank and the other to filter, for example rank by LHR but drop any pair whose PMI falls below a threshold. This keeps LHR’s ordering while using PMI to veto the frequent-but-loose pairs, trading some recall for precision.

Choosing a threshold and indexing strategy

Once a scoring function is chosen, we pick a threshold and add every bi-gram above it to the vocabulary. The threshold is not a precise cut-off; it trades index size against phrase coverage. Missing a bi-gram is not fatal: the document is still retrievable through its individual terms, so recall is unaffected. What we lose is the precision and ranking boost the phrase token would have given, which lets more incidental co-occurrences through as noise. For many use cases, such as fact-checking or broad information search, that extra noise is acceptable.

Building the bi-gram set with nltk (optional reading)

`nltk` provides collocation finders that do the ranking and filtering directly. The result feeds the `bigrams` set used above.

```
from nltk.collocations import BigramCollocationFinder, BigramAssocMeasures
from nltk.corpus import stopwords

finder = BigramCollocationFinder.from_words(tokens)
finder.apply_freq_filter(3)                              # require at least 3 co-
occurrences

ignored = set(stopwords.words('english'))
finder.apply_word_filter(lambda w: len(w) < 3 or w.lower() in ignored)

# take the top-k scoring pairs and keep them as tuples
top = finder.score_ngrams(BigramAssocMeasures.likelihood_ratio)[:200]
bigrams = {pair for pair, score in top}
```

Applying the vocabulary changes how we tokenize, without changing the single scan through the text. Once the accepted bi-grams are known, we tokenize each document, and every query the same way, in one left-to-right pass. At each position we emit the single token, and we do one extra vocabulary lookup to check whether the current token together with the next one forms an accepted bi-gram; if it does, we emit the phrase token as well and move on to the next position, so overlapping phrases are still caught. Keeping the single tokens alongside the phrase is deliberate, as the chapter opener showed: a query for “Holmes” must still match a document that only contains “Sherlock Holmes”.

A compact implementation keeps two vocabularies: the ordinary single tokens, and a set of accepted token pairs stored as Python tuples.

```
bigrams = {("new", "york"), ("york", "city")}   # accepted pairs from PMI/LHR (see
code above)

def apply_phrases(tokens: list[str], bigrams: set[tuple[str, str]]) -> list[str]:
    result = []
    for i, tok in enumerate(tokens):
        result.append(tok)                               # always keep the single
token
        if i + 1 < len(tokens) and (tok, tokens[i + 1]) in bigrams:
            result.append(f"{tok}_{tokens[i + 1]}")      # add the phrase token
    return result

apply_phrases("the city of new york city".split(), bigrams)
# ['the', 'city', 'of', 'new', 'new_york', 'york', 'york_city', 'city']
```

The two overlapping phrases “new york” and “york city” are both emitted, next to the single tokens, exactly the mix the chapter opener described. The accepted-pair set itself comes from the collocation finder: rank with PMI or LHR, apply the frequency and stop-word filters, take the top-k, and store each surviving bi-gram as a tuple.

The construction extends to tri-grams and quad-grams: keep a set of triples, check a window of three tokens, and emit a three-word phrase when it matches. Returns diminish quickly, though. Any four-word sequence is rare enough that the scoring functions lose statistical power, and the extra vocabulary rarely earns its storage. Most production systems stop at bi-grams and handle the occasional longer phrase with proximity search at query time. `nltk` also provides `TrigramCollocationFinder` and `QuadgramCollocationFinder` with matching `TrigramAssocMeasures` and `QuadgramAssocMeasures`.

Compounds

A phrase glues two tokens together. A compound goes the other way: it hides several words inside a single token. German makes this productive: “Wolkenkratzer” (skyscraper) combines “Wolke” (cloud) and “Kratzer” (scratcher). “Rindfleischetikettierungsüberwachungsaufgabenübertragungsgesetz” (a Mecklenburg-Vorpommern law from 1999-2013 delegating the supervision of beef labelling) chains together seven concepts as one token. Finnish and Dutch have the same construction. Programming languages have their own version: `word_tokenize`, `assertEquals`, and `QueryParser` each pack a whole phrase into one identifier.

Compounds break retrieval in an asymmetric way. A user who types “Etikettierung Gesetz” cannot match a document indexed under the full compound. The reverse also fails: a query for “Rindfleischetikettierungsüberwachungsaufgabenübertragungsgesetz” only finds documents that spelled it exactly the same way, which is optimistic for a 63-letter word. The fix is to index both the compound and its parts, giving the retriever a chance to match either.

Endocentric vs exocentric compounds

Not every compound splits usefully. **Endocentric** compounds derive their meaning from their parts: “sunglasses” are a kind of glasses, “Abfalleimer” is a kind of Eimer (bucket), and splitting into parts is safe. **Exocentric** compounds do not: a “skyscraper” is not a kind of “scraper”, and the German “Wolkenkratzer” is not a kind of “Kratzer” (scratcher). Splitting an exocentric compound into parts adds tokens with the wrong semantics to the document, hurting precision. Whether the recall gain outweighs the precision loss depends on the collection: news search benefits from splitting, legal search often does not.

Splitting compounds

Compound splitting proceeds in two stages: generate candidate splits, then score them.

For **generating candidates**, we use language-specific rules. In English, hyphens and hyphenation syllables give the splits: “must-have” splits into “must” and “have”, “skyscraper” into “sky”, “scrap”, “er”. In German, syllable boundaries following German hyphenation rules produce candidates: “Wolkenkratzer” becomes (“wol”, “ken”, “krat”, “zer”), and every way of grouping consecutive syllables into words yields a candidate split: (wolken, kratzer), (wolken, krat, zer), (wol, kenkratzer), and so on. German also uses binding letters like “s” between compound parts (“Schifffahrtskapitän” needs to be tested both as “Schifffahrt-Kapitän” and “Schifffahrts-Kapitän”), so the candidate generator must try both variants.

For **scoring candidates**, we discard any split containing a piece that is not in the language’s dictionary, then score the survivors by how frequent their parts are. Crucially, the score is not a winner-take-all decision. Just as we indexed both “new york” and its parts, and both the long beef-labelling law and its constituents, we do not have to commit to a single decomposition of a compound: we can keep several plausible splits and index the union of their parts alongside the full compound. The score’s job is to rank plausibility and prune weak candidates, not necessarily to crown one winner.

Key Formula: Compound Split Score

$$\text{score}(S) = \frac{1}{|\,S\,|} \sum_{p_i \in S} \log \frac{\text{tf}(p_i)}{N} \tag{3.6}$$

The average log-frequency of the parts in split S, computed against a reference corpus of N tokens. A higher score means the split is into more common words, so it is a more plausible decomposition. Use it either to take the single best split, or to keep the top few plausible splits.

The intuition: the dictionary filter first removes any split containing a piece that is not a real word, so (“Wol”, “Kenkratzer”) is discarded outright because neither piece is a German word. Among the splits that survive, the score ranks plausibility by the frequency of the parts. We then either keep the single top split,

or, favouring recall, keep several plausible splits and index the union of their parts. The average is used rather than the sum so that splits with different numbers of parts are comparable.

Example: Splitting Autobahnausfahrt

Candidate splits of "Autobahnausfahrt" (motorway exit), scored by the average frequency of their parts. Frequencies are on the Zipf scale, where higher means more common and 0 means the word does not appear in the corpus:

Split	Parts (Zipf)	Average	Keep?
(Auto, Bahn, Ausfahrt)	5.3, 5.0, 3.7	4.67	yes
(Autobahn, Ausfahrt)	4.4, 3.7	4.05	yes
(Auto, Bahnausfahrt)	5.3, 0.0	•	discarded ("Bahnausfahrt" unseen)

Two things stand out. First, the score correctly throws out (Auto, Bahnausfahrt) because "Bahnausfahrt" never appears in the corpus. Second, plain average frequency ranks the finer split (Auto, Bahn, Ausfahrt) *above* the more natural (Autobahn, Ausfahrt), because "Auto" and "Bahn" are extremely common words. A frequency score alone tends to over-split into short common words; production splitters counteract this with a penalty for more or shorter parts, or a list of known compounds.

But we do not have to choose between the two surviving splits. We can index "Autobahnausfahrt" together with the union of their plausible parts, "Auto", "Bahn", "Autobahn", and "Ausfahrt", so a query for any of them matches. This over-indexing costs storage and lets some noise through, favouring recall; keeping only the single best split is the extra investment we make when a use case needs the precision.

Where this sits in modern retrieval

Phrase detection and compound splitting are both still active production techniques. Lucene ships two compound-word filters. The `HyphenationCompoundWordTokenFilter` is the two-stage method above in production form: a hyphenation grammar, the same pattern file that breaks words across lines at the end of a text line, proposes candidate break points in place of our syllable rules, and a dictionary keeps only the fragments that are real words. The matching sub-words are added to the token stream next to the original compound, exactly the union-indexing we described. Its sibling, the `DictionaryCompoundWordTokenFilter`, skips the hyphenation step and tests substrings directly against the word list, which is slower and produces more spurious fragments, so Elasticsearch recommends the hyphenation variant for German, Dutch, and other compounding languages. Solr exposes the same filters through the Redlink extension, and actively maintained dictionaries such as `uschindler/german-decompounder` provide the word lists these filters consume.

Dense retrieval learns the compound structure implicitly through sub-word tokenizers like BPE and WordPiece, but the learning is imperfect. A 2023 study (arXiv:2305.14214) showed that SentencePiece, the tokenizer most widely used with multilingual language models, still splits German compounds into semantically incoherent fragments a significant fraction of the time. Hybrid retrieval stacks that combine BM25 with dense embeddings therefore keep the explicit decompounder on the BM25 branch, letting the neural side learn what it can while the classical side handles the compounds the classical way.

Phrase detection has an analogous story. Bi-gram indexing appears in modern lexical retrievers wherever named entities matter: product-search stacks index brand-name bi-grams, patent search indexes technical compounds, and academic search indexes conference names. PMI and LHR remain the standard offline scoring functions for building those bi-gram vocabularies.

Hands-on: Phrases and Compounds

Extract PMI and LHR bi-grams from a text corpus, apply frequency and stop-word filters, and split a set of German compounds into their parts.

> Open notebook ->
>
> *Includes pre-run results; you can read through or download and experiment.*

The next section leaves the document-recall problem behind and turns to the query side of the two-way failure from the chapter opener. Given a natural-language query, how do we extract enough structure from it that the retriever can do more than bag-of-words matching?

Query Understanding

The first three sections closed the recall gap on the document side. A well-processed collection now has one token per concept, phrases identified, and compounds split. The chapter opener's other failure remains: the query "Bücher von Goethe" arrives as three tokens and the retriever has no idea that "Bücher" and "books" are the same concept, that "Goethe" names a person, or that the sentence is a request for works by a specific author. This section adds three tools that extract that kind of structure from free-text: word-relationship expansion, part-of-speech tagging, and named entity recognition. The section ends with the practical concern of spelling: a query that arrives misspelled needs to be repaired before any of these tools can help.

Word relationships: synonyms and homonyms

Stemming and lemmatization collapse different surface forms of the *same* word onto one token. They do nothing about the looser assumption we used so far: that one normalized token is independent of another normalized token. Every retrieval model in the book to this point has relied on it, treating tokens as unrelated dimensions with no notion that two of them might mean similar things or that one might mean two things. Real language breaks that assumption in two directions. A **synonym** is two different words for the same concept ("buy" and "purchase"); a **homonym** is one word carrying two concepts ("bank" the institution and "bank" the riverside). Lemmatization helps with neither, because "buy" and "purchase" reduce to unrelated stems and "bank" keeps a single stem whichever sense is meant. This section handles both with classical lexical tools; the semantic search chapter returns to the independence assumption in full and replaces it with learned representations that place related tokens near each other.

Synonyms break recall. A user searching a product catalog for "purchase history" misses documents indexed under "buy" or "buying", even though they hold exactly what the user wants, purely because the tokens differ. Two responses:

- **Synonym expansion (classical)**. Attach a synonym list to each token. When the document contains "purchase", also index "buy" and "acquire". When the query contains "buy", also search for "purchase" and "acquire". The synonym lists come from curated resources (WordNet for English, thesaurus databases for other languages) or from domain-specific glossaries (medical terminologies, legal dictionaries).
- **Dense embeddings (modern)**. Instead of maintaining synonym lists, learn a vector space where "buy" and "purchase" naturally end up near each other. This is the approach of the semantic search chapter and it subsumes the synonym problem, at the cost of an embedding model and a vector index.

Domain vocabularies show the same problem more sharply. The chapter opener's F1 query, "Who won the F1 race on the weekend?", needs to match documents that call the same event a "Grand Prix" and never use the tokens "F1" or "race" at all. WordNet does not capture this mapping because it is not a linguistic synonym but a domain shorthand. The fix is a domain-specific synonym list maintained by the search team: `F1` maps to `{Formula 1, Formula One, Grand Prix}`, `race` maps to `{Grand Prix, GP}`. Product-search systems maintain similar lists for brand aliases ("iPhone" -> "Apple iPhone", "iPhones") and category synonyms ("laptop" -> "notebook").

Hand-maintained lists work well for the common, high-value terms a search team can anticipate, but they cannot keep up with the long tail: driver names the list has never seen, one-off event aliases, or code words specific to a community. This is where dense retrieval starts to pay for itself, because an embedding model learns which words appear in similar contexts without any manual list. Modern production stacks combine both: hand-crafted lists for the head, embeddings for the tail. The full argument comes back in the semantic search chapter.

Homonyms are the other direction, and the harder one. A query for "bank" cannot know whether the user means the financial institution or the riverside, and "lead" could be the metal or the verb meaning to guide. Two responses again:

- **Context from surrounding words (classical)**. The words around a homonym usually fix its sense. Part-of-speech tagging is the simplest case: "lead" the noun (the metal) and "lead" the verb (to guide)

carry different tags, so tagging at index and query time separates them. Broader lexical context helps further: a text that also mentions “money”, “account”, or “loan” points to the financial “bank”, while “river”, “water”, or “shore” points to the riverside. This works whenever there is enough surrounding text to read.

- **Contextual embeddings (modern)**. Sentence encoders and large language models represent a word by the company it keeps, so “bank” in “the river bank” and “bank” in “the savings bank” receive different vectors. They resolve the sense from context without any hand-written rules, which is why they handle homonyms far more reliably than lexical rules do. This is the approach developed in the semantic search chapter.

Both approaches lean on context, so both fail when the query is too short to supply any: a bare query “bank” gives the system nothing to disambiguate on. There the only options are to return results for both senses or to ask, which is what web search does with “did you mean” suggestions. Genuine ambiguity with no context is inherent to language, and no technique fully removes it.

Hypernyms and hyponyms

WordNet provides two more relationships that expand queries in a different direction. A **hypernym** is a broader category (“animal” is a hypernym of “cat”), and a **hyponym** is a narrower one (“cat” is a hyponym of “animal”, and “mammal” is both a hyponym of “animal” and a hypernym of “cat”). These relationships form a hierarchy, and every English noun in WordNet sits somewhere in that hierarchy.

Three retrieval uses:

- **Faceted search**. A user searching for “animals” is offered “cats”, “dogs”, “birds” as sub-facets that drill down into the hypernym tree. If the initial query returns too few results, the user is offered the parent facet (“mammals”) that broadens it. Amazon’s category tree and Wikipedia’s category system are both hypernym hierarchies dressed up as navigation UIs.
- **Query expansion with weighted hyponyms**. A search for “cat” is silently expanded to include hyponyms like “kitten”, “tabby”, “siamese”, each with a lower weight than the original term. Documents about specific cat breeds surface for a general “cat” query.
- **Relevance ranking with hypernym paths**. Instead of expanding the query, the scoring function is adjusted so a document mentioning “labrador” contributes something to a query for “dog” even without an explicit expansion.

```
from nltk.corpus import wordnet

for synset in wordnet.synsets('cat', pos='n')[:3]:
    print(synset.name(), '->', synset.definition())
    print('  hypernyms:', [h.name() for h in synset.hypernyms()])
    print('  hyponyms: ', [h.name() for h in synset.hyponyms()[:3]])

# cat.n.01 -> feline mammal usually having thick soft fur and no ability to roar
#   hypernyms: ['feline.n.01']
#   hyponyms:  ['domestic_cat.n.01', 'wildcat.n.03']
```

WordNet covers English; multilingual equivalents such as Open Multilingual WordNet exist for around 30 languages but with less coverage than the English version.

Part-of-speech tagging

Sentences are made of words that fall into grammatical classes: noun, verb, adjective, adverb, pronoun, article, and so on. A **part-of-speech tagger** assigns one of these labels to each token in a sentence, using surrounding words to disambiguate cases where the same surface form belongs to different classes (“run” as a noun in “a good run” versus “run” as a verb in “I run daily”). Figure 3.1 shows a POS-tagged parse tree for a simple English sentence, with the POS tags at the leaves and phrase-level structure above them.

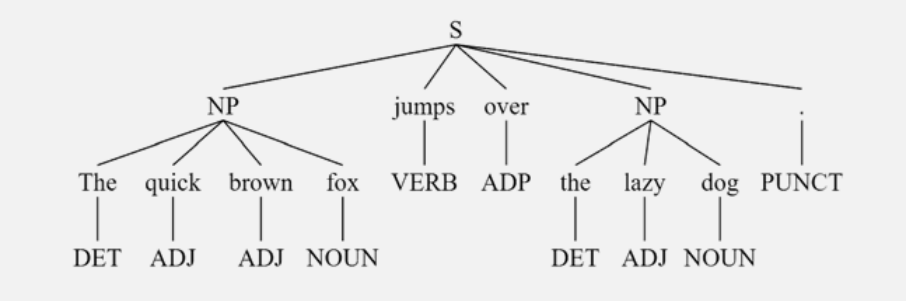


Figure 3.1: Constituency parse tree for "The quick brown fox jumps over the lazy dog." POS tags (DET, ADJ, NOUN, VERB, ADP, PUNCT) appear at the leaves; phrase-level constituents such as NP (noun phrase) sit at intermediate nodes.

POS tagging supports three retrieval-side uses:

- **Selective stop-word filtering**. Not every occurrence of "it" is a pronoun. In the phrase "the IT department", "IT" is a noun and should be kept; in "it is easy", it should be dropped. POS-aware stop-word filtering keeps content-bearing occurrences and removes function-word ones. This is how the "IT security" case from the stop-word discussion in the previous section is handled correctly.
- **Lemmatizer disambiguation**. WordNet's lemmatizer needs the POS tag: `wordnet.lemmatize("meeting", "n")` returns `"meeting"` while `wordnet.lemmatize("meeting", "v")` returns `"meet"`. Feeding the lemmatizer the wrong tag produces the wrong lemma.
- **Question-form analysis**. The query "Who is Albert Einstein?" has a WH-word ("who"), a copula verb ("is"), and a proper noun ("Albert Einstein"). Combining that structure with the recognition that "Albert Einstein" is a person's name (see the next subsection) tells the retriever this is a person-lookup query, and the right response is not a keyword search but a lookup in a people database or a redirect to Wikipedia. The intent-routing section returns to this example.

Three families of taggers have been used historically:

- **Rule-based taggers** apply hand-written rules based on suffixes and surrounding words. Fast and simple, but each language needs its own rule set and rules cannot easily be extended.
- **Statistical (HMM) taggers** model the sentence as a sequence of hidden POS states emitting observed tokens. Transition and emission probabilities are learned from a POS-tagged corpus; decoding uses the Viterbi algorithm. Dominant approach from the 1990s to the mid-2010s.
- **Neural taggers** run a small transformer or LSTM over the token sequence and output a POS tag per token. This is the current default in `spaCy` and in most Hugging Face pipelines.

All three families reach around 97-99% accuracy on English news text. On informal genres (social media, product reviews) the neural taggers pull ahead because they handle out-of-vocabulary words better.

> **See Also**
>
> The Hidden Markov Model formulation used by statistical taggers is developed in the ML foundations appendix, alongside the Viterbi decoding algorithm. This section does not repeat the HMM math.

```python
import spacy
nlp = spacy.load('en_core_web_sm')

doc = nlp("Who is Albert Einstein?")
for token in doc:
    print(f"{token.text:12s} {token.pos_:8s} {token.tag_:6s} {token.dep_}")

# Who          PRON     WP     nsubj
# is           AUX      VBZ    ROOT
# Albert       PROPN    NNP    compound
# Einstein     PROPN    NNP    attr
# ?            PUNCT    .      punct
```

Two tag sets appear in practice. The `pos_` field uses the coarse Universal tag set (around 17 categories: NOUN, VERB, ADJ, ADV, PROPN, PRON, DET, ADP, ...). The `tag_` field uses the fine-grained Penn

Treebank tag set (around 45 categories: VBZ for third-person singular verb, NNP for proper noun, WP for WH-pronoun). The Universal tags are enough for retrieval-side filtering; the Penn tags are needed for building parse trees.

Named entity recognition

The tokens "Albert Einstein" in the previous example are not just two proper nouns; they name a specific person, and the retrieval system that recognizes them as such can act on that recognition. **Named entity recognition (NER)** classifies spans of tokens into entity types: person, location, organization, product, date, money, and so on. NER is a natural extension of POS tagging: it runs after tokenization and POS tagging (or jointly with them in modern neural pipelines) and outputs entity spans.

The approaches behind NER mirror the POS taggers above. The earliest systems were rule-based, combining gazetteers (lists of known people, places, and organizations) with hand-written patterns such as a capitalized word following a title like "Dr." or preceding a suffix like "Inc.". Statistical sequence models replaced them: a conditional random field (CRF) labels each token from features of the token and its neighbours, trained on annotated corpora such as CoNLL-2003. Modern systems fine-tune a transformer for token classification, tagging each sub-word with its entity type. This is the current state of the art and what spaCy's larger models and Hugging Face NER pipelines use.

```
doc = nlp("Jack Higgins deposits £50,000 with BestBank in London.")
for ent in doc.ents:
    print(f"{ent.text:15s} {ent.label_}")

# Jack Higgins    PERSON
# £50,000         MONEY
# BestBank        ORG
# London          GPE
```

For query understanding, four categories dominate the retrieval-side use cases:

- **Person names**. "Who is Albert Einstein?" should trigger a person-lookup: prioritize Wikipedia, IMDb, MusicBrainz, and other person-oriented sources rather than running a keyword search across the whole web.
- **Time and date**. "Who won the F1 race last weekend?" contains a temporal expression. Retrieval should restrict to news articles from the past few days and rank recent items above older ones.
- **Locations**. "What to do in Basel?" should prioritize regional content and augment the results with a map. The location is a hard filter, not just another keyword.
- **Product brands**. "Where can I buy the latest iPhone?" should boost shopping and product-review pages, and can trigger a price-comparison widget.

Every NER category exposes a routing decision. The classifier and router are the subject of the next section; this section only extracts the entities themselves.

NER also generates candidate phrases automatically: consecutive tokens tagged as `PERSON` produce a bi-gram or tri-gram like "Albert Einstein" that should be indexed as a unit, exactly as in the previous section's phrase-detection discussion. This is often faster and more targeted than PMI or LHR scoring over the whole corpus, because named entities are exactly the phrases that matter for entity-oriented search.

For tooling, spaCy and Stanza provide NER for many languages out of the box, nltk offers a lighter `ne_chunk`, and the Hugging Face `token-classification` pipeline exposes fine-tuned transformer models such as `dslim/bert-base-NER`. Beyond libraries, the major cloud providers offer NER as a managed API: Amazon Comprehend, Google Cloud Natural Language, and Azure AI Language all return typed entities (often alongside key phrases and relations) without training a model. This convenience is why they anchor many production document-processing pipelines, extracting parties and amounts from contracts, fields from invoices, or names and dates from scanned forms.

Chunking with rule-based grammars

For queries that do not fit neatly into a POS-plus-NER analysis, a lightweight **chunker** groups consecutive tokens matching a small grammar. The classic pattern is:

```
NP -> DET? ADJ* NOUN
```

which matches a noun phrase like “a red car” or “the lazy dog”. Chunking gives the retriever access to noun-phrase-level tokens without running a full parser. `nltk.RegexpParser` implements this cheaply:

```
grammar = r"NP: {<DT>?<JJ>*<NN>}"
chunker = nltk.RegexpParser(grammar)
tree = chunker.parse(nltk.pos_tag(nltk.word_tokenize("a red car")))
```

More elaborate grammars produce dependency parses that connect the noun phrase “red car” to the adjective “red” that modifies it. Stanford CoreNLP and spaCy’s dependency parser produce these for free once POS tags are available. For most retrieval scenarios noun-phrase chunking is enough.

Spell correction and “did you mean?”

Every query understanding pipeline has to handle typos. A user typing “Bücher von Goehte” (transposed letters) gets no results if the retriever demands an exact string match on “Goethe”, regardless of how good the rest of the pipeline is. The classical response is a two-part strategy: correct at both indexing time and query time.

- **At indexing time**, keep the original token but also add the auto-corrected form. A document containing the misspelling “Goehte” is indexed under both “Goehte” and “Goethe”, so users find it whether they type the correct or the incorrect spelling.
- **At query time**, run the query through the spell-checker. If a token is not in the dictionary, offer corrections and either search all of them silently or ask “Did you mean ‘Goethe’?”. Modern search engines do both: they auto-run the corrected query and let the user click back to the original spelling if the correction was wrong.

Spell correction is especially tricky for names. “Britney” is often misspelled as “Britny”, “Brittney”, or “Britnee”, which argues for folding them all back to “Britney”. The catch is that several of these forms are themselves legitimate names that other people genuinely carry, so a single variant can be at once a typo of one name and the correct spelling of another. A spell-checker that rewrites every variant to one canonical “Britney” then merges distinct people and loses anyone who is really spelled the other way. This is the “did you mean Britney, or Britnie?” problem. The pragmatic compromise is to keep the original spelling in the index and, at query time, expand to the common variants rather than silently rewriting to a single form.

Two standard algorithms underpin classical spell correction:

- **Edit distance** (Damerau-Levenshtein). Score candidate corrections by the minimum number of insertions, deletions, substitutions, and transpositions to reach the query. Widely used but slow to compute exhaustively; production systems restrict the candidate set with a phonetic prefilter or a small edit-radius trie.
- **Phonetic codes** (Soundex, Metaphone). Reduce each word to a compact code that captures its rough pronunciation. Different spellings of the same-sounding word share a code and are candidates for one another. Essential for name variants that no edit-distance approach can handle.

Query expansion and query rewriting

Several of the transformations described so far share a family resemblance. Synonym mapping, hypernym expansion, spell correction, alias substitution, and temporal normalization all take the tokens the user typed and change them so the resulting query matches more of the collection. In the IR literature these transformations fall under two umbrella names:

- **Query expansion** adds tokens to the original query without removing the ones the user typed. Synonym expansion, weighted-hyponym expansion, and adding all common spelling variants of a

name to a query all fall here. The original query terms usually stay at full weight and the added terms come in at a lower weight, so a document that matches the user's exact words still ranks above one that matches only via expansion.
- **Query rewriting** transforms the query text itself, replacing or restructuring tokens. Spell correction ("Goehte" -> "Goethe"), alias substitution ("F1" -> "Formula 1"), temporal normalization ("last weekend" -> a date range), and compound splitting on the query side ("Wolkenkratzer" -> "Wolken", "Kratzer") are query rewrites. The rewritten query goes to the retriever; the original may or may not be preserved for logging.

The line between the two blurs in practice. A rewrite that adds tokens alongside the originals (`F1 OR "Formula 1" OR "Grand Prix"`) is an expansion in disguise. An expansion that later drops the original terms in favour of the added ones is a rewrite. What matters is that the retriever sees a different query from the one the user typed, and both directions are standard tools.

This chapter has been building both throughout:

- **Section 1** — language detection informs both expansion (which stop-word and synonym list to use) and rewriting (which stemmer to apply). Case, Unicode, and accent normalization are rewrites at the character level.
- **Section 2** — stemming and lemmatization are rewrites at the token level ("carried" -> "carri" for Porter, "carried" -> "carry" for WordNet).
- **Section 3** — compound splitting rewrites one token into several on both the document side and the query side.
- **Section 4** — synonym and hypernym mappings are expansions; spell correction is a rewrite.
- **Section 5** — temporal normalization and domain-alias substitution are rewrites applied during intent routing.

Two techniques not covered here belong in the same family and are worth naming so students recognize them elsewhere:

- **Pseudo-relevance feedback**, also called blind feedback. This is the relevance-feedback and query-expansion mechanism of the Binary Independence Model from Probabilistic Ranking and BM25, run without any user judgements. Instead of asking the user which results are relevant, it assumes the top-k retrieved documents are relevant and applies the same term-discrimination weighting to choose expansion terms, the high-scoring terms like "woodland" in that chapter's worked example. It is a retrieval-model technique rather than a text-processing one, which is why it belongs with vector-space and BM25 ranking.
- **LLM-based query rewriting**. A language model reformulates the query. Common patterns include HyDE (write a plausible answer and search for that instead of the question), multi-query (generate several rewrites and union their results), and step-back prompting (generalize the query one level before searching). These belong to the retrieval-augmented generation chapter.

The chapter opener's F1 query is a plain example of query rewriting. "Who won the F1 race on the weekend?" is rewritten by dropping the WH-pronoun "who" and the verb "won" as non-content, expanding "F1" and "race" to "Formula 1" and "Grand Prix", and turning "the weekend" into a date range. What reaches the retriever is a scoped keyword query rather than a question, and the next section walks that rewrite through the full routing pipeline. Classical rewriting stops there: it can retrieve the right race-report page but cannot produce the short answer "Verstappen". That last step needs a reader or generator to extract the answer from the retrieved text which is covered in a later chapter (retrieval-augmented generation).

Where this sits in modern retrieval

Every classical technique in this section still runs somewhere in a modern search stack. Web search engines apply POS tagging, NER, and spell correction to every query before any retrieval or ranking. Product-search systems for e-commerce use NER heavily to extract brand names, category constraints, and numerical facets ("under $50"). Enterprise search over document management systems relies on synonym expansion using domain-specific terminologies.

The one classical technique that dense embeddings largely subsume is synonym expansion: a well-trained embedding model puts “purchase” and “buy” near each other in vector space, so the query “purchase history” retrieves documents about “buying” without an explicit synonym list. Hypernym reasoning is partially learned but less reliably: an embedding model may or may not know that a labrador is a dog, depending on the training data. POS tagging, NER, and spell correction remain explicit steps even in fully neural pipelines because they produce structured outputs (tags, spans, corrected strings) that downstream systems can act on discretely. Large language models can substitute for all of them in a single prompt, but for high-throughput retrieval the classical stack is still cheaper by orders of magnitude.

Hands-on: Query Understanding

Run POS tagging and NER on a set of natural-language queries, extract candidate entities, and compare the `spaCy` and `nltk` outputs.

Open notebook ->

Includes pre-run results; you can read through or download and experiment.

The final section takes everything this section produces (language ID, POS tags, entities, corrections) and turns it into a routing decision: which backend should the query go to?

Intent Routing and Classification

Recall the second query from the chapter opener: “Bücher von Goethe”. By this point in the chapter, the pipeline has enough machinery to see it clearly. The tokenizer splits it into three tokens. The rule-based language detector recognizes German characters and function words. POS tagging identifies “Bücher” as a plural noun, “von” as a preposition, and “Goethe” as a proper noun. Named entity recognition promotes “Goethe” to a person entity. What has not been decided yet is what to do with all this structure. Should the query go to a book catalogue, a web index, an author database, or a general-purpose LLM? This section turns the extracted features into that routing decision.

The end-to-end query pipeline

A query enters the retrieval system as a raw string. Before any matching happens, it flows through the classical pipeline this chapter has been building:

```
raw query
    -> tokenize                        (section 1)
    -> normalize (case, Unicode)       (section 1)
    -> segment sentences               (section 1, if multi-sentence)
    -> detect language                 (section 1 rules + this section's classifier)
    -> stop-word filter                (section 2, language-specific)
    -> stem or lemmatize               (section 2, language-specific)
    -> split compounds                 (section 3, if applicable)
    -> POS-tag                         (section 4)
    -> extract named entities          (section 4)
    -> spell-correct                   (section 4)
    -> classify intent                 (this section)
    -> route to backend                (this section)
```

The order matters. Language detection has to come before any language-specific step; POS tagging has to come before selective stop-word filtering; NER has to come before intent classification because the entity types are input features to the classifier. What comes out at the end is a structured representation of the query that a downstream system can act on: a language, a set of entities, an intent label, and a corrected token list.

Two of these classification steps use the same underlying machinery. Language detection classifies a query into one of the languages the system knows; intent classification classifies a query into one of the backends the system supports. Both are text classification problems and both can be solved with the same Naive Bayes model. This section develops that model once, then applies it to both problems.

Naive Bayes for text classification

Naive Bayes chooses the most probable class given the observed features by applying Bayes’ theorem and assuming that features are conditionally independent given the class. The derivation lives in the ML foundations appendix; here we state only the decision rule.

For a feature vector $\boldsymbol{x} = (x_1, \ldots, x_M)$ and classes $C_1, \ldots, C_K$:

Key Formula: Naive Bayes Decision Rule

$$\hat{C} = \arg\max_k P(C_k) \cdot \prod_{j=1}^{M} P\left(x_j \texttt{ mid } C_k\right) \tag{3.7}$$

In log form, which is what implementations actually compute:

$$\hat{C} = \arg\max_k \log P(C_k) + \sum_{j=1}^{M} \log P\left(x_j \texttt{ mid } C_k\right) \tag{3.8}$$

The prior $P(C_k)$ captures the base rate of each class, and the likelihood $P\left(x_j \texttt{ mid } C_k\right)$ captures how well feature x_j fits class C_k. The full derivation from Bayes’ theorem is in the appendix.

Three things need to be learned from a labelled training corpus:

- **The prior $P(C_k)$**: fraction of training documents in class C_k. When the training data reflects real-world class frequencies, use it directly; when it is artificially balanced (equal examples per class) or when class balance is unknown, set a uniform prior $P(C_k) = 1/K$ and the priors drop out of the argmax.
- **The likelihood $P(x_j \texttt{ mid } C_k)$**: fraction of feature occurrences in class C_k that fall on token or feature x_j. This is a maximum-likelihood estimate over the training data.
- **A smoothing constant**. If x_j never appeared in the training data for class C_k, the maximum-likelihood estimate is zero, and one zero collapses the whole product to zero. Add-one (Laplace) smoothing avoids this by adding a small constant to every count.

The product in the decision rule is a direct consequence of the independence assumption: taking the features to be conditionally independent given the class turns the joint likelihood $P(\boldsymbol{x} \texttt{ mid } C_k)$ into the product $\prod_j P(x_j \texttt{ mid } C_k)$ of per-feature terms. This is what makes the estimation possible at all. In classical text classification the feature space has tens or hundreds of thousands of dimensions, so any particular full vector $\boldsymbol{x}$ almost never recurs in the training data, and $P(\boldsymbol{x} \texttt{ mid } C_k)$ could never be estimated directly. Each individual dimension x_j, by contrast, is observed many times across the corpus, so the per-feature factors $P(x_j \texttt{ mid } C_k)$ can be counted reliably. Independence trades a joint probability we cannot measure for a product of marginals we can.

The assumption is not literally true, since tokens in real text are not independent, but Naive Bayes still performs well on text because the errors it introduces tend to affect all classes similarly and rarely change which class wins the argmax. Its speed and small memory footprint are the reasons it stays in production stacks two decades after neural classifiers became viable.

Language detection as classification

Language detection uses character-based n-grams as features. For n from 1 to 5, count how many times each n-gram appears in the input text. This produces a bag-of-n-grams representation exactly like the bag-of-words representation used for topic classification, except that the tokens are short character strings instead of whole words.

Character n-grams work well as features because each language has characteristic short sequences that rarely appear in other languages:

- German trigrams: `sch`, `cht`, `hen`, `und`, `der`, `die`
- English trigrams: `the`, `ing`, `and`, `ion`, `ent`
- French trigrams: `les`, `des`, `que`, `ent`, `oui`
- Italian trigrams: `che`, `non`, `per`, `ent`

Some trigrams like `ent` appear in multiple languages but with different frequencies, which is enough for a Naive Bayes classifier to disambiguate.

Setting up the classifier:

- **Classes**: the target languages. `lingua` covers 70 languages; a domain-specific detector might cover only 5 or 10.
- **Features**: character n-grams from a per-language frequency profile built during training. Most of the discriminating signal sits in a modest set of frequent n-grams, so a frugal detector can store only those significant likelihoods and stay tiny. `lingua` in high accuracy mode keeps the full observed profile across five n-gram orders (unigram through fivegram): the English model holds about 380,000 n-grams (∼3.3 MB), German about 470,000 (∼4.2 MB). It trades size for robustness on short and ambiguous inputs rather than pruning to the discriminating core.
- **Prior**: uniform when the incoming text has unknown language distribution; empirical when the retrieval system knows its user base is 60% English, 30% German, 10% French.
- **Likelihood**: for the multinomial variant, $P(t_j \texttt{ mid } C_k) = n_{k,j} / \sum_l n_{k,l}$ where $n_{k,j}$ is the count of n-gram j in the training text for language k. With +1 smoothing, $P(t_j \texttt{ mid } C_k) = (n_{k,j} + 1) / \left(\sum_l n_{k,l} + V\right)$ where V is the vocabulary size.

The decision rule then takes the standard log-form:

$$\hat{C} = \arg\max_k \left(\log P(C_k) + \sum_j c_j \log P(t_j \texttt{ mid } C_k) \right) \tag{3.9}$$

where c_j is the observed count of n-gram j in the input.

Example: Detecting German for "Bücher von Goethe"

Extract the character trigrams, padding each word boundary with _ so that word-initial and word-final patterns become features in their own right (the classic Cavnar-Trenkle text-categorization convention):

```
_bü, büc, üch, che, her, er_, _vo, von, on_, _go, goe, oet, eth, the, he_
```

For each candidate language $k \in \{\text{DE}, \text{EN}, \text{FR}, \text{IT}\}$, compute $\log P(C_k) + \sum \log P(t_j \texttt{ mid } C_k)$ over the 15 trigrams. German wins by a large margin because:

- `_bü`, `büc`, `üch` are common in German and essentially absent from English, French, and Italian (ü is a diacritic that only German and a few other Latin-script languages use).
- `_vo`, `von`, `on_` are common in German ("von" is the fourth most frequent German preposition).
- `_go`, `goe`, `oet` appear in German names (Goethe, Görlitz, Göttingen) at rates much higher than in English.

The boundary padding matters because languages differ in how words *begin* and *end*, not only in their interior sequences: `_bü` and `_vo` capture a distinctly German way of starting a word. `lingua` reaches the same verdict from unpadded n-grams taken within each word (its stored profiles carry no boundary marker), returning `Language.GERMAN` with a confidence of 0.96.

For very short queries with only one or two trigrams that discriminate between languages, confidence drops. `lingua` reports its confidence as a normalized posterior, and the retrieval system can fall back to a default language (English, or the user's browser locale) when the top confidence is too close to the second-best.

```python
from lingua import Language, LanguageDetectorBuilder

detector = LanguageDetectorBuilder.from_all_languages().build()
detector.detect_language_of("Bücher von Goethe")
# Language.GERMAN

# Restrict to expected languages and get confidence scores
euro = [Language.ENGLISH, Language.GERMAN, Language.FRENCH, Language.ITALIAN]
d2 = LanguageDetectorBuilder.from_languages(*euro).build()
d2.compute_language_confidence_values("Bücher von Goethe")
# GERMAN: 0.96, ENGLISH: 0.02, FRENCH: 0.01, ITALIAN: 0.01
```

`langdetect` is a lighter alternative: rule- and n-gram-based, 55 languages, ISO-code output.

```python
from langdetect import detect
detect("Bücher von Goethe")   # 'de'
detect("Livres de Goethe")    # 'fr'
detect("Books by Goethe")     # 'en'
```

Intent routing as classification

The same machinery routes the query to a backend. Once the language is known, an **intent classifier** decides which of the search system's supported intents the query expresses:

- `book_search`: query looks for books (title, author, ISBN)
- `web_search`: general-purpose keyword search
- `image_search`: query looks for images

- `product_search`: query looks for shoppable products
- `people_search`: query looks for information about a person
- `news_search`: query has a temporal component or a current-events subject
- `map_search`: query looks for a location or directions
- `weather`, `calculator`, `definition`, `translation`: one-shot vertical intents

The training data is query logs annotated with the backend the user actually clicked into. Features are richer than for language detection because more of the pipeline's output can be used:

- The token bag after stemming and stop-word removal
- POS-tag counts (how many nouns, verbs, WH-words)
- NER labels present in the query (has-PERSON, has-LOCATION, has-DATE, has-MONEY)
- Question-form markers (starts with a WH-word, ends with a question mark)
- Detected language (as a categorical feature)

With these features, "Bücher von Goethe" scores highly on `book_search` because it contains a book-domain plural noun ("Bücher") and a PERSON entity ("Goethe") in an author-attribution construction. "What to do in Basel?" scores highly on `map_search` and `web_search` because it contains a WH-word, a LOCATION entity, and no product or person entity. "Who is Albert Einstein?" scores highly on `people_search` because of the WH-word combined with a PERSON entity.

The classifier itself is exactly the same Naive Bayes model as for language detection, just with different features and different classes. In practice, a modern production stack fine-tunes a small neural classifier on top of the same features because it captures conjunctions ("has-PERSON AND WH-word AND is-question") that Naive Bayes cannot express under the independence assumption. In a later chapter, we study the neural variants using LLM to make the routing decision (agentic RAG).

> **Naive Bayes prefers uniform priors for query classification**
>
> Query-log training data reflects the current backend distribution, not the ideal one. If 80% of past queries went to web search, an empirical prior will route new queries to web search 80% of the time regardless of their features, because the prior term overwhelms weak features. For intent routing, use a uniform prior and let the features drive the decision, or use empirical priors only after balancing the training data across intents.

End-to-end walkthrough

Putting all five sections of the chapter together on the running query:

```
Input:              "Bücher von Goethe"

Tokenize:           ['Bücher', 'von', 'Goethe']
Normalize (case):   ['bücher', 'von', 'goethe']
Detect language:    German (confidence 0.96)
Stop words:         drop 'von'  ->  ['bücher', 'goethe']
Stem (Snowball DE): ['buch', 'goeth']
POS tag:            [(Bücher, NOUN), (Goethe, PROPN)]
NER:                [(Goethe, PERSON)]
Spell-check:        no correction needed
Intent classify:    book_search  (features: language=DE, PERSON present,
                                  plural NOUN of book-domain lemma)
Route:              library catalog with filters
                        language = "de"
                        author   = "Goethe"
                        content  = *
```

The library catalog answers a structured query with all Goethe titles it holds. No general-purpose keyword match against "Bücher von Goethe" was ever needed.

A second walkthrough on the other chapter-opener query. This one combined a WH-question form, tokens that would not appear in the answer, and a domain-vocabulary gap.

```
Input:                        "Who won the F1 race on the weekend?"

Tokenize:                     ['Who', 'won', 'the', 'F1', 'race', 'on',
                               'the', 'weekend', '?']
Normalize (case):             ['who', 'won', 'the', 'f1', 'race', 'on',
                               'the', 'weekend']
Detect language:              English
POS tag:                      [(who, WH-PRON), (won, VERB), (the, DET),
                               (F1, PROPN), (race, NOUN), (on, ADP),
                               (the, DET), (weekend, NOUN)]
NER:                          [(F1, EVENT), (the weekend, DATE)]
Selective stop-word filter:   drop 'the', 'on'
                              keep 'who' as a classification feature
                                   (WH-word signals question form)
Stem/lemma (English):         ['win', 'F1', 'race', 'weekend']
Synonym expansion (domain):   F1     -> {Formula 1, Grand Prix}
                              race   -> {Grand Prix, GP}
Temporal normalization:       the weekend -> [Sat, Sun of the past weekend]
                                          -> ['2026-08-08', '2026-08-09']
Intent classify:              news_search
                                  features: WH-word + EVENT entity +
                                            DATE entity + motorsport terms
                                  features against 'question' subclass:
                                            WH-word + won + who
Route:                        news index with
                                  query = "Formula 1" OR "Grand Prix" OR "GP"
                                  date  = [2026-08-08, 2026-08-09]
                                  sort  = date desc
```

The classical pipeline turned three tokens of question form and one date reference into a scoped keyword query the news index can answer: a Boolean OR over motorsport aliases, a date filter for the past weekend, and a sort by date. Each of the three gaps named in the chapter opener has a specific step above that addresses it. The missing “who” is absorbed into the classification features rather than searched as a keyword. The missing “weekend” becomes a temporal range that a race-results page dated in that range will match. The missing “race” is replaced by the domain-synonym expansion.

That is as far as this chapter takes the F1 example. What the news backend returns is a ranked list of race-result pages, not the two-word answer “Verstappen won” that the user actually requested. Two later chapters carry the example the rest of the way:

- The **semantic search** chapter shows how a dense-retrieval index closes the domain-vocabulary gap without a hand-maintained synonym list, catching the driver names, sponsor terms, and circuit aliases the classical stack never saw.
- The **retrieval-augmented generation** chapter runs a language model over the top-ranked page and returns the extracted name as the answer, turning a list of results into a direct answer.

The classical pipeline of this chapter is still there in both cases. It prepares the query and prunes the search space so the more expensive downstream steps only see the pages that matter. The router itself does not do the retrieval; it picks a backend and passes the structured query to it. Everything upstream in this chapter was building the structure that the router now dispatches on.

Where this sits in modern retrieval

Classical intent routing runs in every large search stack in 2025. Google Search, Amazon Search, and enterprise search products all have an intent-classification stage before the retriever. The classifier is rarely still Naive Bayes: production systems have moved to gradient-boosted trees or small transformers because the feature interactions matter and the classifier is on the query-latency critical path only in a few-millisecond budget. Naive Bayes remains the pedagogical baseline and a common first pass in cost-sensitive deployments (mobile, embedded).

Large language models can perform the whole pipeline in one prompt: extract entities, detect the language, classify the intent, and generate the structured backend query. This is a live area of production adoption, especially for open-ended assistant systems. The trade-offs against classical pipelines are latency (an LLM call is 100-1000 milliseconds versus sub-millisecond for the classical stack), cost (per-query LLM cost is measurable, per-query classical cost is not), and confidence quantification (Naive Bayes gives calibrated posteriors, LLMs do not). Production stacks in 2025 usually run the classical pipeline first, escalate to LLMs for the queries where the classical stack's confidence is low, and reserve LLMs for high-value verticals like agentic search and complex reformulation.

Two forward references pick up where this section leaves off:

- The text classification chapter covers deep-learning text classification with TextCNN and transformer-based classifiers, using sentiment analysis as its running example. The Naive Bayes classifier of this section is the baseline those methods are compared against.
- The semantic-search chapter covers embedding-based intent classification: instead of extracting features from the query and running Naive Bayes, embed the query into a vector space and compare it to prototype vectors for each intent. Modern hybrid systems combine both approaches.

Hands-on: Intent Routing

Train a Naive Bayes language detector on a small multi-language corpus, then extend it into an intent classifier over a set of example queries, and walk a query through the full routing pipeline.

Open notebook ->

Includes pre-run results; you can read through or download and experiment.

The next page summarizes the chapter and points to the material that follows it in the book.

Summary

Method Comparison

Stemmers and lemmatizers side by side.

Method	Approach	Correctness	Speed	Multi-lingual	Needs dictionary
Porter	Rule-based suffix stripping (English)	pseudo-stem	fast	English only	no
Lancaster	Aggressive rule-based (English)	pseudo-stem, over-stems short words	fastest	English only	no
Snowball	Rule-based framework	pseudo-stem	fast	20+ languages	no
WordNet	Dictionary + POS lemmatizer	linguistic lemma	slow	English only (multilingual variants weaker)	yes
spaCy	Neural POS tagger + dictionary lemmatizer	linguistic lemma	medium	many languages	yes

Use a rule-based stemmer when performance dominates or when no dictionary is available for the language. Use a dictionary-based lemmatizer when accuracy on strongly-inflected languages matters (German case-inflected nouns, English strong verbs like "went -> go") or when the downstream task needs linguistic base forms rather than arbitrary reductions. Snowball is the go-to compromise for multi-language corpora when only rule-based options are practical.

Key Takeaways

1. Classical retrieval fails in two directions: one concept has many surface forms on the document side (recall gap), and free-text queries carry structure that the retriever ignores (query understanding gap). Advanced text processing addresses both.
2. Tokenization is the first stage and the largest source of quality issues. A naive regex tokenizer breaks on possessives, numbers, abbreviations, and punctuation. Modern nltk and spaCy tokenizers handle these but still need retrieval-oriented cleanup.
3. Normalization decides which surface forms collapse to the same token. Case folding is nearly always applied. Unicode normalization is applied silently. Accent folding trades recall for precision and depends on the scenario.
4. Sentence segmentation is a small but essential step. Every RAG chunking strategy, every POS tagger, and every query-analysis pipeline depends on being able to find sentence boundaries reliably.
5. Stop-word removal is a size-versus-recall trade-off. Modern BM25-based systems handle stop words gracefully through IDF and can afford to keep them in the index. Legacy vector-space systems remove them aggressively. Phrase queries and titles like Stephen King's "It" break under aggressive removal.
6. Stemming is a fast pseudo-linguistic reduction; lemmatization is a slower dictionary-based one that produces the true linguistic base form. Choose based on speed budget, language complexity, and whether downstream steps need real words.
7. Phrases and compounds sit on opposite sides of the word-boundary problem. Bi-grams like "New York" pack multiple tokens into one concept and need explicit phrase indexing; PMI and LHR are the classical selectors for which bi-grams are worth adding. Compounds like "Bücherregal" do the reverse, hiding multiple concepts in one token, and are split into their parts by log-frequency scoring, though only endocentric compounds split safely (a "Bücherregal" is a kind of "Regal"; a "Wolkenkratzer" is not a kind of "Kratzer", so splitting it hurts precision).

8. Query understanding turns free text into structured features: language, POS tags, named entities, corrected spelling. Together they let the retriever route the query to the right backend and generate structured queries rather than doing bag-of-words matching.
9. A Naive Bayes classifier applied to hand-engineered features is enough to build both a language detector and an intent classifier. Modern production stacks use neural classifiers for accuracy but keep Naive Bayes as the baseline and as the fast path for cost-sensitive deployments.
10. Every classical technique in this chapter still runs in production 2025 search stacks. Dense retrieval subsumes some of them (synonym expansion) implicitly, but hybrid retrieval combines both branches and needs the classical stack on the lexical side.

Key Formulas

$$\operatorname{pmi}(t_1, t_2) = \log_2 \frac{N \cdot \operatorname{tf}(t_1, t_2)}{\operatorname{tf}(t_1) \cdot \operatorname{tf}(t_2)} \tag{3.10}$$

Pointwise Mutual Information: high when two tokens almost always appear together and rarely apart. Biased toward rare-word bi-grams; requires a minimum-frequency filter.

$$\log \lambda = \log \frac{L(H_1)}{L(H_2)} \tag{3.11}$$

Log Likelihood Ratio: log-ratio of the probability under independence to the probability under dependence. Robust on sparse data; ranks by $-2 \log \lambda$.

$$\operatorname{score}(S) = \frac{1}{\mid S \mid} \sum_{p_i \in S} \log \frac{\operatorname{tf}(p_i)}{N} \tag{3.12}$$

Compound Split Score: average log-frequency of the parts in split S. Choose the split with the highest score.

$$\hat{C} = \arg\max_k \left(\log P(C_k) + \sum_{j=1}^{M} \log P(x_j \texttt{ mid } C_k) \right) \tag{3.13}$$

Naive Bayes Maximum A Posteriori: pick the class that maximizes the log-prior plus the sum of log-likelihoods. Used for language detection over character n-grams and for intent classification over feature vectors combining tokens, POS tags, and NER labels.

Exam focus

- The two failure directions classical retrieval hits, and which chapter technique addresses each
- Why possessives, abbreviations, and multi-token numbers break the naive regex tokenizer
- The difference between stemming (pseudo-stem, rule-based) and lemmatization (real lemma, dictionary-based), and when to choose each
- Why aggressive stop-word removal breaks queries like “The Who” and “IT security”, and how BM25’s IDF weighting reduces the need for aggressive removal
- How PMI and LHR rank the same bi-grams differently and why both are useful
- Why endocentric compounds split usefully and exocentric ones do not
- The three retrieval-side uses of POS tagging (stop-word filtering, lemmatizer disambiguation, question-form analysis)
- How language detection and intent classification are the same Naive Bayes machinery with different features and classes
- Where in modern hybrid retrieval each classical step still runs, and where dense embeddings subsume it

Self-Check Questions

1. (Understand) The naive tokenizer produces `['Watson', 's']` from “Watson’s”. Under what retrieval scenario is this desirable, and under what scenario is it a problem? What information is permanently lost?
2. (Understand) Why does BM25 tolerate stop words that appear in the index better than a raw vector-space TF-IDF model does?

3. (Analyze) Given the *A Study in Scarlet* corpus where "said" occurs 207 times, "Holmes" 98 times, "Sherlock" 52 times, "Sherlock Holmes" 52 times, and "said Holmes" 12 times, compute PMI for the two bi-grams "Sherlock Holmes" and "said Holmes". Explain why the ranking matches (or does not match) what a human would call "the more meaningful phrase". Assume a corpus of $N = 43,200$ tokens.
4. (Analyze) Explain why aggressive accent folding hurts a legal-document retrieval system for German but helps a general-purpose web search engine over the same language.
5. (Analyze) A retrieval system indexes "Wolkenkratzer" only as itself (no compound splitting). A user queries "Wolke". A different user queries "Kratzer". Which query, if any, retrieves documents about skyscrapers, and why? Would splitting help both users equally? Would you split this specific compound?
6. (Evaluate) A production search team proposes replacing the entire classical query pipeline (tokenization, stemming, POS, NER, intent classification) with a single call to a large language model that outputs a structured query directly. State two reasons the team might keep the classical pipeline anyway, and one query type where the LLM call is likely to win.
7. (Apply) A user queries "Livres de Molière". Walk through the end-to-end pipeline from section 5, listing the output of each step and the final routing decision. Assume the search stack supports French.

Test Your Knowledge

Take the Chapter 3 Quiz ->

Further Reading

- Porter, M. F. (1980). **An Algorithm for Suffix Stripping**. *Program*, 14(3), 130-137. Author copy. The original Porter Stemmer paper, still the reference for how to build a rule-based English stemmer and the ancestor of every rule-based stemmer that followed.
- Paice, C. D. (1990). **Another Stemmer**. *ACM SIGIR Forum*, 24(3), 56-61. Introduced the Lancaster stemmer and the aggressive suffix-stripping strategy that trades precision for speed.
- Manning, C. D., & Schütze, H. (1999). **Foundations of Statistical Natural Language Processing**, Chapter 5: Collocations. *MIT Press*. The standard reference for PMI, likelihood-ratio testing, and other statistical collocation-scoring methods; source of the LHR formulation used in this chapter.
- Kiss, T., & Strunk, J. (2006). **Unsupervised Multilingual Sentence Boundary Detection**. *Computational Linguistics*, 32(4), 485-525. Free PDF. The Punkt algorithm, still the sentence-segmentation default in NLTK almost two decades later.
- Koehn, P., & Knight, K. (2003). **Empirical Methods for Compound Splitting**. *Proceedings of the 10th Conference of the European Chapter of the ACL*, 187-193. Free PDF. The frequency-based compound-splitting method this chapter uses, evaluated for machine translation but applicable directly to retrieval.
- Minixhofer, B., Pfeiffer, J., & Vulić, I. (2023). **CompoundPiece: Evaluating and Improving Decompounding Performance of Language Models**. arXiv:2305.14214. Shows that modern subword tokenizers (SentencePiece, BPE) still decompound German poorly, motivating why explicit decompounders remain in hybrid retrieval stacks.

Implementations and tools

- NLTK — Python toolkit for classical NLP; ships all the stemmers, tokenizers, POS taggers, and collocation finders used in this chapter.
- spaCy — Industrial-strength NLP library with neural taggers and lemmatizers for many languages.
- WordNet — Lexical database for English synonyms, hypernyms, and hyponyms.
- Snowball — Multi-language stemmer framework, source of the German and French stemmers used in section 2.
- lingua-language-detector — Character-n-gram Naive Bayes language detector covering 70+ languages.
- Elasticsearch decompounder documentation — Production compound-splitter configuration for German, Dutch, and other compounding languages.

The classical tools in this chapter sit at the end of a seventy-year research arc. The optional note below traces that history from 1950s symbolic AI through statistical corpus methods to today's neural models, for readers who want the wider context.

From ELIZA to Transformers: a short history of NLP (optional reading)

The classical techniques in this chapter descend from a much longer research programme that runs from 1950s symbolic AI through statistical corpus-based methods to today's neural language models. Not exam-relevant, but useful for placing the tools in context and understanding where the field went next.

Symbolic era (1950s-1980s). Rule-based systems dominated NLP. Joseph Weizenbaum's ELIZA (1966) simulated a Rogerian psychotherapist by matching keyword patterns and substituting into templates (Figure 3.2); users routinely believed it understood them. Terry Winograd's SHRDLU (1971) parsed natural-language commands in a blocks-world simulator (Figure 3.3), resolving pronouns ("it") and detecting ambiguity ("which pyramid?") explicitly. Both systems worked spectacularly in their narrow domains and failed the moment inputs strayed outside those domains. The lesson was that natural language cannot be fully formalized with hand-written rules.

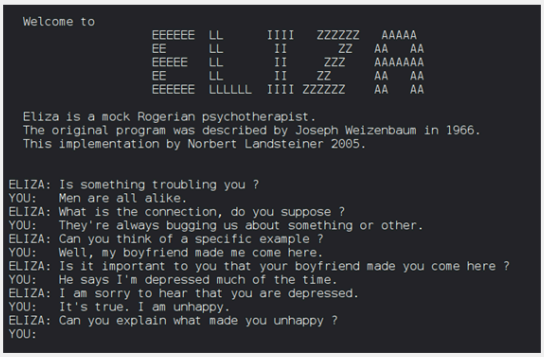


Figure 3.2: A terminal session with ELIZA (Landsteiner's 2005 reimplementation of Weizenbaum's 1966 program). Simple pattern matching, no world model, and users still developed emotional attachments.

This is an example of a **SHRDLU** session:

Person: Pick up a big red block.
Computer: OK.
Person: Grasp the pyramid.
Computer: I DON'T UNDERSTAND WHICH PYRAMID YOU MEAN.
Person (changing their mind): Find a block which is taller than the one you are holding and put it into the box.
Computer: BY "IT", I ASSUME YOU MEAN THE BLOCK WHICH IS TALLER THAN THE ONE I AM HOLDING.
Computer: OK.
Person: What does the box contain?
Computer: THE BLUE PYRAMID AND THE BLUE BLOCK.
Person: What is the pyramid supported by?
Computer: THE BOX.
Person: How many blocks are not in the box?
Computer: FOUR OF THEM.

Figure 3.3: A SHRDLU session (Winograd, 1971), tracking pronoun references across turns and reporting ambiguity explicitly when a referring expression is underspecified.

Statistical era (1980s-2010s). Hidden Markov Models drove advances in POS tagging, speech recognition, and named entity recognition. N-gram language models powered early machine translation and text generation. Annotated corpora such as Brown and the Penn Treebank provided the labelled data these methods required. The statistical bet paid off: data-driven models beat hand-crafted rules on almost every task. The limits were the size of the labelled datasets, heavy feature engineering, and no ability to capture long-range meaning.

Neural era (2010s-present). Bengio's 2003 neural language model represented words as continuous vectors. Word2Vec (Mikolov et al., 2013) made word embeddings practical at scale. RNNs, then LSTMs and GRUs, handled sequence dependencies. In 2017 the Transformer replaced recurrence with self-attention, enabling parallel processing of full sequences and much longer contexts. The techniques in this chapter still power the lexical-retrieval side of modern hybrid systems; the neural side is covered in the semantic-search chapter.